Domain 1 – Snowflake AI Data Cloud Features & Architecture

Complete Beginner's Master Guide | COF-C03 2026

Technical + Layman Definitions | Edge Cases | Hands-On SQL | 20 Questions Per Topic | Full Answer Key

> How to use this guide
>
> 1. Read the **Layman Explanation** first — build the mental model
> 2. Read the **Technical Explanation** — understand precise mechanics, edge cases, and exam traps
> 3. Run every **Hands-On Query block** in your Snowflake free trial (sign up: https://signup.snowflake.com)
> 4. Attempt all **20 questions** per topic WITHOUT looking at answers
> 5. Check the **Answer Key** at the very bottom of this document
>
> **Exam facts:** Domain 1 = **31% of exam weight** (biggest domain). 100 questions total. 115 minutes. Pass = 750/1000 scaled score. $175 USD.[1][2]

SUBDOMAIN 1.1 — Snowflake Architecture: The Three Layers

Layman's Explanation

Imagine building a traditional database is like buying a house: the kitchen (compute) and the pantry (storage) are permanently attached to each other in the same building. When you want more pantry space, you have to renovate the whole kitchen too. When you want a bigger kitchen, you have to build a bigger house even if your pantry is mostly empty.

Snowflake builds a different kind of house — one where the pantry is in one cloud, the kitchens are rented on-demand, and a smart building manager handles security, scheduling, and efficiency.

The three layers are:

- **Storage Layer** = An infinite pantry in the cloud (Amazon S3 / Azure Blob / Google Cloud Storage). Your data lives here 24/7, even when nothing is cooking.
- **Compute Layer** = Rental kitchens (Virtual Warehouses). You only pay while chefs are actively cooking. Turn them off; billing stops.

- **Cloud Services Layer** = The invisible building manager. Handles authentication, decides the best recipe (query optimization), tracks what's in every shelf (metadata), and enforces who can enter which room (access control).

The magic: **these three layers are completely independent.** Growing storage doesn't change compute cost. Adding a 10th virtual warehouse doesn't copy the data 10 times. This is why Snowflake is fundamentally different from anything that came before.

Technical Explanation

Architecture Classification

Snowflake uses a **hybrid architecture** that is a combination of two older approaches:[3][4]

Architecture	Old Approach	Snowflake's Version
Shared-Disk	All compute nodes share one storage pool	Snowflake storage is shared across all warehouses
Shared-Nothing	Each node has its own private data	Each warehouse is an independent MPP cluster that doesn't interfere with others

This hybrid gives Snowflake the **data consistency** of shared-disk and the **independent scaling** of shared-nothing. No other architecture offered both simultaneously before Snowflake.

Layer 1: Database Storage Layer[4][3]

Physical location: Cloud object storage — Amazon S3, Azure Blob Storage, or Google Cloud Storage — depending on which cloud region your Snowflake account is deployed in.

Data format: All data is stored in **micro-partitions** — immutable, compressed, columnar files:

- Size: 50–500 MB uncompressed (typically ~100 MB compressed)
- Format: Columnar (each column's data stored together, not row-by-row)
- Encryption: AES-256 at rest (always on, always automatic)
- Immutability: Micro-partitions are never modified in-place — DML operations create new partitions
- Format type: **Snowflake's proprietary format** — NOT Parquet, NOT ORC, NOT Avro (critical exam trap)

What the storage layer stores:

- Table data (in micro-partitions)
- Query result cache (in Cloud Services metadata — NOT in storage layer itself)
- Time Travel historical partitions
- Fail-Safe retained partitions

Key behaviors:

- Storage billing continues 24/7 regardless of warehouse activity
- Users NEVER directly access storage files — all access is mediated by Snowflake
- Snowflake automatically selects the best compression algorithm per column
- No manual partitioning, indexing, sorting, or vacuum needed

Layer 2: Query Processing Layer (Virtual Warehouses)[5][3]

What it is: A named cluster of cloud VMs (virtual machines) that executes SQL queries.

How it works:

1. Query arrives → Cloud Services parses + optimizes it → sends execution plan to the active warehouse
2. Warehouse nodes read required micro-partitions from storage → cache them on local NVMe SSD
3. Each node processes its portion in parallel (MPP = Massively Parallel Processing)
4. Results are aggregated → returned to the client

Key behaviors:

- Warehouses are fully **independent** — Warehouse A and Warehouse B can run against the same data simultaneously with zero conflicts
- Billing is **per-second** with a **60-second minimum per start event**
- Warehouse state: `STARTED` (billing runs), `SUSPENDED` (billing stops), `RESIZING`
- Nodes are **deallocated** on suspend → **local NVMe cache is lost**
- Resuming takes ~1–5 seconds (new VMs provisioned from cloud provider pool)

Layer 3: Cloud Services Layer[6][3][^4]

The **always-on brain** of Snowflake. Never billed for a warehouse because it IS a managed service, not a warehouse.

Responsibilities:

Sub-Component	What It Does
Authentication Service	Validates passwords, MFA tokens, OAuth tokens, SAML assertions, key-pairs
Authorization (RBAC)	Checks role privileges before any operation executes
Query Parser	Tokenizes and parses SQL text into an Abstract Syntax Tree
Query Optimizer	Builds a cost-based execution plan (which partitions to scan, join order, etc.)

Sub-Component	What It Does
Execution Coordinator	Distributes the plan to warehouse nodes
Metadata Store	Tracks every object: schema definitions, column stats, micro-partition metadata (min/max/count/null per column per partition), clustering info
Transaction Manager	Implements MVCC (Multi-Version Concurrency Control) for ACID compliance
Query Result Cache	Stores exact result sets of recent queries for up to 24 hours
Infrastructure Manager	Provisions/deprovisions warehouse VMs on demand

Billing edge case: Cloud Services is free up to **10% of daily compute credits**. Only excess above 10% is billed. For most accounts, Cloud Services effectively costs nothing.[^1]

Metadata-only queries: Because the Cloud Services metadata store knows the row count, min value, and max value of every column in every micro-partition, certain queries can be answered **without a running warehouse**:

- `SELECT COUNT(*) FROM table` ✅ (row count is in metadata)
- `SELECT MIN(col), MAX(col) FROM table` ✅ (min/max per column is in metadata for non-null columns)
- `SELECT SUM(col)`, `AVG(col)`, `SELECT * WHERE filter` ❌ (must scan data → need warehouse)

⚠ Critical Exam Traps for 1.1

Trap	Correct Answer
"What format does Snowflake use for storage files?"	Snowflake's **proprietary** columnar format — NOT Parquet/ORC
"How does Snowflake differ from Shared-Nothing architectures?"	Storage is **shared** (not per-node); only compute is isolated
"Who manages file compression and encryption?"	**Snowflake** does it automatically — customers don't configure it
"Does dropping a warehouse delete data?"	**No** — storage and compute are completely independent
"What happens to Cloud Services when all warehouses are suspended?"	Cloud Services **keeps running** — it's always on
"Can multiple warehouses access the same table simultaneously?"	**Yes** — no storage-level read locking

Hands-On Queries: Architecture Exploration

```
-- ============================================================
-- SETUP: Log into your Snowflake trial account at app.snowflake.com
-- Open a new Worksheet in Snowsight and run these one at a time
-- ============================================================

-- 1. Discover your account context
SELECT CURRENT_USER()       AS my_user;
SELECT CURRENT_ROLE()       AS my_role;
SELECT CURRENT_ACCOUNT()    AS account_id;
SELECT CURRENT_REGION()     AS cloud_region;  -- e.g., AWS_US_EAST_1
SELECT CURRENT_VERSION()    AS snowflake_version;
SELECT CURRENT_TIMESTAMP()  AS utc_time;

-- 2. See all three layers in one query result:
--    - Cloud Services: parsed and planned this query (always happening)
--    - Storage: SNOWFLAKE_SAMPLE_DATA is a real dataset in storage
--    - Compute: depends on query type
SELECT
    'Cloud Services'  AS layer,
    'Parsed + optimized this query' AS role
UNION ALL SELECT 'Storage', '~6M rows of TPCH order data in micro-partitions'
UNION ALL SELECT 'Compute', 'Used only if query scans data files';

-- 3. LAYER 1 DEMO — Metadata-only query (NO warehouse needed!)
--    Suspend your warehouse first to prove it works without compute:
ALTER WAREHOUSE IF EXISTS compute_wh SUSPEND;
SELECT CURRENT_WAREHOUSE();  -- Returns NULL

SELECT COUNT(*) FROM SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.ORDERS;
-- SUCCESS with zero credits! Row count lives in Cloud Services metadata.

SELECT MIN(O_ORDERDATE), MAX(O_ORDERDATE)
FROM SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.ORDERS;
-- SUCCESS! Min/Max on columns are stored in partition metadata.

-- This FAILS without a warehouse (needs to scan actual data values):
SELECT SUM(O_TOTALPRICE) FROM SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.ORDERS;
-- Error: "No active warehouse selected in the current session"

-- 4. LAYER 2 DEMO — Create and control a virtual warehouse
USE ROLE SYSADMIN;

CREATE WAREHOUSE IF NOT EXISTS learn_wh
    WAREHOUSE_SIZE = 'X-SMALL'    -- 1 credit/hr — cheapest!
    AUTO_SUSPEND   = 120          -- Stop after 2 min idle
    AUTO_RESUME    = TRUE         -- Start automatically on query
    COMMENT        = 'COF-C03 learning warehouse';

USE WAREHOUSE learn_wh;
SELECT CURRENT_WAREHOUSE();  -- Now shows: LEARN_WH

-- Now SUM works because we have a warehouse:
SELECT SUM(O_TOTALPRICE) AS total_revenue
FROM SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.ORDERS;
```

```
-- 5. LAYER 3 DEMO — Observe Cloud Services metadata
SHOW DATABASES;     -- Cloud Services metadata (no warehouse needed)
SHOW WAREHOUSES;    -- Cloud Services metadata (no warehouse needed)
SHOW TABLES IN DATABASE SNOWFLAKE_SAMPLE_DATA;  -- no warehouse needed

-- 6. Prove multiple warehouses can access same data simultaneously
CREATE WAREHOUSE IF NOT EXISTS learn_wh_2
    WAREHOUSE_SIZE = 'X-SMALL'
    AUTO_SUSPEND   = 60
    AUTO_RESUME    = TRUE;

-- Both warehouses target the same table — no conflict!
-- (In real life you'd open two sessions; here we alternate to demonstrate)
USE WAREHOUSE learn_wh;
SELECT COUNT(*) FROM SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.LINEITEM;

USE WAREHOUSE learn_wh_2;
SELECT MIN(L_SHIPDATE) FROM SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.LINEITEM;
-- Both work on the same storage — no locks, no copies!

-- 7. Verify storage is independent of warehouses
--    Drop a warehouse — does data disappear?
DROP WAREHOUSE IF EXISTS learn_wh_2;

-- Data is still here!
SELECT COUNT(*) FROM SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.LINEITEM;
-- Still returns millions of rows — dropping a warehouse NEVER touches data.

-- 8. Architecture confirmed in ACCOUNT_USAGE
USE ROLE ACCOUNTADMIN;
SELECT WAREHOUSE_NAME, START_TIME, END_TIME, CREDITS_USED
FROM SNOWFLAKE.ACCOUNT_USAGE.WAREHOUSE_METERING_HISTORY
WHERE START_TIME >= DATEADD('hour', -2, CURRENT_TIMESTAMP())
ORDER BY START_TIME DESC;
-- See exactly when billing started/stopped — perfectly matching STARTED/SUSPENDED s
```

Topic 1.1 — 20 Practice Questions

Q1. Snowflake is described as a "hybrid" architecture. What two classic architectures does it combine?

- A) Shared-disk and shared-nothing
- B) Shared-memory and distributed-file-system
- C) Master-slave and peer-to-peer
- D) Cloud-native and on-premise

Q2. Where does Snowflake physically store your table data?

- A) On dedicated Snowflake-owned servers in its own private data centers
- B) On NVMe SSD disks inside Virtual Warehouses

- C) In cloud object storage (Amazon S3, Azure Blob, or Google Cloud Storage) depending on deployment region
- D) In the Cloud Services metadata store

Q3. What file format does Snowflake use internally to store data in micro-partitions?

- A) Apache Parquet
- B) Apache ORC
- C) Apache Avro
- D) Snowflake's own proprietary columnar format

Q4. A company suspends ALL their Virtual Warehouses at 5 PM. Which of the following stop billing at that point? *(Select ALL that apply)*

- A) Virtual Warehouse compute credits
- B) Storage costs (data at rest)
- C) Cloud Services charges for metadata queries
- D) None — Snowflake charges a flat monthly fee regardless

Q5. You run `SELECT COUNT(*) FROM my_table` with no active warehouse. What happens?

- A) Error — all SELECT statements require an active warehouse
- B) The query auto-starts a warehouse, runs, then stops it
- C) The query succeeds — row count is available from Cloud Services metadata
- D) The query runs but returns only an approximate count

Q6. Five different Virtual Warehouses all query the same table simultaneously. How many physical copies of the table data exist?

- A) 5 copies — one per warehouse
- B) 1 copy — all warehouses share the same storage layer
- C) 2 copies — Snowflake keeps an active copy and a backup
- D) It depends on the warehouse sizes

Q7. What does MPP stand for, and why does it matter for Virtual Warehouses?

- A) Managed Processing Protocol — handles network routing
- B) Massively Parallel Processing — multiple warehouse nodes work on different parts of a query simultaneously to speed it up
- C) Memory-Partitioned Performance — memory is partitioned across nodes
- D) Multi-Provider Platform — enables cross-cloud execution

Q8. What is the Cloud Services layer primarily responsible for? *(Select ALL that apply)*

- A) Physically storing user data in columnar files
- B) Query parsing, optimization, and execution planning
- C) Authentication and authorization (RBAC enforcement)
- D) Metadata management (tracking schemas, column stats, partition info)
- E) Running SQL computations (aggregations, joins)

Q9. A developer drops a Virtual Warehouse. What happens to the data stored in the associated tables?

- A) Data is permanently deleted
- B) Data is moved to Fail-Safe storage for 7 days, then deleted
- C) Nothing — data lives in the Storage layer, which is completely independent from Virtual Warehouses
- D) Data is archived to the default stage

Q10. Which query can be answered from Cloud Services metadata WITHOUT a running Virtual Warehouse?

- A) `SELECT SUM(revenue) FROM sales`
- B) `SELECT AVG(salary) FROM employees`
- C) `SELECT COUNT(*) FROM customers`
- D) `SELECT * FROM orders WHERE status = 'PENDING'`

Q11. AES-256 encryption "at rest" in Snowflake means:

- A) Only sensitive columns are encrypted; others use plain text
- B) Data is encrypted while being transferred between storage and warehouses
- C) All data stored in Snowflake's storage layer is permanently encrypted using AES-256, always and automatically
- D) Users must opt-in to encryption per table

Q12. What happens to a Virtual Warehouse's local NVMe SSD cache when it is SUSPENDED?

- A) It is saved to cloud storage for the next startup
- B) It is completely lost — all cached micro-partitions are evicted
- C) It is compressed and held in Cloud Services for 24 hours
- D) It persists for 7 days, matching the Fail-Safe period

Q13. Which cloud providers does Snowflake natively support for deployment? *(Select ALL that apply)*

- A) Amazon Web Services (AWS)
- B) Microsoft Azure

- C) Google Cloud Platform (GCP)
- D) IBM Cloud
- E) Oracle Cloud

Q14. True or False: The Cloud Services layer requires at least one active Virtual Warehouse to function.

- A) True — Cloud Services depends on warehouse compute to run
- B) False — Cloud Services runs continuously as a managed service, independent of any warehouse
- C) True — but only the SYSTEM warehouse (auto-created) keeps it running
- D) False — but Cloud Services becomes read-only when no warehouse is active

Q15. Cloud Services billing is triggered when its usage exceeds what daily threshold?

- A) 5% of daily compute credits
- B) 10% of daily compute credits
- C) 25% of daily compute credits
- D) Cloud Services is always free — never billed

Q16. A query requires `SELECT SUM(price) FROM products`. No warehouse is active. Why does this fail, when `SELECT COUNT(*) FROM products` would succeed?

- A) SUM requires ACCOUNTADMIN role to run
- B) SUM requires scanning ALL actual price values — not stored in metadata. COUNT(*) uses only the row count, which IS in metadata.
- C) SUM only works on temporary tables
- D) COUNT(*) requires a warehouse too — the premise of the question is wrong

Q17. What is MVCC and which Snowflake layer implements it?

- A) Multi-Value Column Compression — implemented in the Storage Layer
- B) Multi-Version Concurrency Control — implemented in the Cloud Services Layer to allow concurrent reads/writes without blocking
- C) Managed Virtual Cluster Configuration — implemented in the Compute Layer
- D) Multi-Volume Cache Control — implemented in both Storage and Compute

Q18. Snowflake's storage layer uses what data orientation inside micro-partitions?

- A) Row-based — each row is stored as a contiguous record
- B) Columnar — each column's data is stored together, enabling efficient column-only reads
- C) Hybrid row-column — rows for small tables, columns for large tables

- D) JSON-based — each row is stored as a JSON document

Q19. Which of the following statements are TRUE about Snowflake micro-partitions? *(Select ALL that apply)*

- A) They range from 50–500 MB of uncompressed data
- B) Users can manually assign rows to specific micro-partitions
- C) They are immutable — DML operations create new partitions rather than modifying existing ones
- D) Each partition stores metadata: min value, max value, count, and null count per column
- E) Micro-partitions are stored in Apache Parquet format

Q20. A company has 1 TB of data in Snowflake and runs 50 concurrent users on their reporting warehouse. They also run overnight ETL jobs. Which architectural benefit of Snowflake allows the ETL and reporting workloads to be completely isolated from each other?

- A) Multi-cluster warehouses automatically route traffic
- B) The separation of storage and compute — you can create two independent warehouses (one for reporting, one for ETL) that both read from the same storage without interference
- C) Snowflake's built-in query scheduler separates ETL and reporting
- D) The Cloud Services layer partitions query traffic by role

SUBDOMAIN 1.2 — Snowflake Interfaces and Tools

Layman's Explanation

Snowflake gives you several "doorways" to enter and interact with the platform:

Snowsight = The main front door — a modern web app in your browser. No installation needed. Point, click, type SQL, see results, build charts. This is where beginners start.

Snowflake CLI = The back door for power users. A command-line tool you run from your terminal. Great for scripting, automation, and CI/CD pipelines. Type `snow` commands instead of clicking.

IDE Integrations = Third-party doors. Tools like VS Code can connect directly to Snowflake via extensions so developers never have to leave their code editor.

The exam tests you on WHAT each tool is for, its KEY features, and WHEN you'd choose one over another.

Technical Explanation

Snowsight (Web Interface)[^7]

What it is: Snowflake's primary, unified, browser-based interface (accessible at `app.snowflake.com`).

Key components:

Feature	Description
Worksheets	SQL or Python editors. Support multiple tabs, version history, keyboard shortcuts. Can run SQL or Python in Notebooks.
Dashboards	Build visual reports by combining charts and tables from saved worksheets
Data tab	Browse databases, schemas, tables, columns without writing SQL
Activity tab	View query history, query profile, execution plans, bytes scanned
Admin tab	Manage users, roles, warehouses, resource monitors, billing
Marketplace	Browse and install Snowflake Marketplace data products
Notebooks	Interactive cell-by-cell Python/SQL execution environment (in-platform)
Streamlit Apps	Host and run Streamlit applications directly inside Snowsight
Monitoring	View warehouse utilization, credit consumption, storage costs

Edge cases / exam traps:

- Snowsight replaced the **Classic Console** (old UI). Classic Console is deprecated. Exam questions may reference it to test if you know which is current.
- **Query Profile** is a Snowsight feature showing the visual execution plan of a query — used for performance tuning. Key stats: partitions scanned, bytes scanned from cache, spill to disk.
- You can share worksheets with other users in your account.
- Snowsight supports **multi-statement queries** in a single worksheet run.

Snowflake CLI[^8]

What it is: A command-line interface tool (`snow` command) for interacting with Snowflake from a terminal, shell script, or CI/CD pipeline.

How to install: `pip install snowflake-cli-labs` (Python-based)

Key command groups:

Command	Purpose
`snow sql -q "SELECT 1"`	Execute SQL from the command line
`snow object list database`	List Snowflake objects
`snow stage list`	List files in a stage
`snow stage put ./file.csv @stage`	Upload files to a stage
`snow streamlit deploy`	Deploy a Streamlit app to Snowflake
`snow cortex summarize --file text.txt`	Run Cortex AI from CLI
`snow notebook execute`	Run a Snowflake Notebook
`snow app run`	Deploy a Snowflake Native App

Configuration: Stored in `~/.snowflake/config.toml` — supports multiple named connections (e.g., dev, prod, staging).

Edge cases:

- The Snowflake CLI replaced the older **SnowSQL** (command-line client). Exam may test both; know that Snowflake CLI is the modern recommended tool.
- SnowSQL is still available and usable, but Snowflake CLI is the current direction.

IDE Integrations[9][8]

Visual Studio Code (VS Code):

- Snowflake extension for VS Code allows running SQL directly from the editor
- Connect using your account credentials, browse objects in a side panel
- Get autocomplete, syntax highlighting for Snowflake SQL

Jupyter Notebooks (via `snowflake-connector-python` or `snowflake-snowpark-python`):

- Connect to Snowflake from local Jupyter Notebooks
- Use Snowpark DataFrame API for Python-native data manipulation

dbt (data build tool):

- Popular transformation layer that generates SQL and runs it against Snowflake
- dbt models become Snowflake tables or views

Other supported integrations: DataGrip, PyCharm, IntelliJ IDEA (all with JDBC drivers).

Hands-On Queries: Exploring Snowsight Features

```
-- ============================================================
-- PART 1: Explore Snowsight Query Features
-- ============================================================

-- Run this and observe the execution details in Snowsight UI:
USE ROLE SYSADMIN;
USE WAREHOUSE learn_wh;
USE DATABASE SNOWFLAKE_SAMPLE_DATA;
USE SCHEMA TPCH_SF1;

-- After running this, click the query in Query History
-- and open "Query Profile" to see the visual execution plan
SELECT
    O_ORDERSTATUS,
    COUNT(*)          AS order_count,
    SUM(O_TOTALPRICE) AS total_value,
    AVG(O_TOTALPRICE) AS avg_value
FROM ORDERS
GROUP BY O_ORDERSTATUS
ORDER BY total_value DESC;

-- In Query Profile you can see:
-- - "Partitions scanned" vs "Partitions total"
-- - "Bytes scanned from cache" (Warehouse cache)
-- - "Bytes scanned from remote storage" (S3/Blob/GCS)
-- - Processing time per node

-- ============================================================
-- PART 2: Context functions — useful in any interface
-- ============================================================
-- These work in Snowsight, CLI, and any IDE connection:
SELECT CURRENT_USER()       AS user;
SELECT CURRENT_ROLE()       AS role;
SELECT CURRENT_DATABASE()   AS db;
SELECT CURRENT_SCHEMA()     AS schema;
SELECT CURRENT_WAREHOUSE()  AS warehouse;
SELECT CURRENT_REGION()     AS region;
SELECT CURRENT_ACCOUNT()    AS account;
SELECT CURRENT_VERSION()    AS sf_version;

-- Get all context in one row:
SELECT
    CURRENT_USER()      AS username,
    CURRENT_ROLE()      AS active_role,
    CURRENT_DATABASE()  AS active_db,
    CURRENT_SCHEMA()    AS active_schema,
    CURRENT_WAREHOUSE() AS active_wh,
    CURRENT_TIMESTAMP() AS session_time;

-- ============================================================
-- PART 3: Multi-statement execution in Snowsight worksheet
-- ============================================================
-- Snowsight supports running multiple statements at once
-- (Run All or highlight specific statements)
```

```
CREATE OR REPLACE DATABASE INTERFACE_DEMO;
USE DATABASE INTERFACE_DEMO;
CREATE SCHEMA demo;
USE SCHEMA demo;

CREATE OR REPLACE TABLE tool_test (
    id      INTEGER AUTOINCREMENT PRIMARY KEY,
    tool    VARCHAR(50),
    purpose VARCHAR(200)
);

INSERT INTO tool_test (tool, purpose) VALUES
    ('Snowsight',     'Web-based UI for SQL, dashboards, admin, monitoring'),
    ('Snowflake CLI', 'Command-line tool for automation and scripting'),
    ('VS Code',       'IDE integration for developers writing SQL and Python'),
    ('dbt',           'SQL-based transformation tool for data pipelines'),
    ('Jupyter',       'Python notebook interface via Snowpark connector');

SELECT * FROM tool_test;

-- ============================================================
-- PART 4: Simulate CLI behavior (what CLI does under the hood)
-- ============================================================
-- The Snowflake CLI runs SQL commands via the REST API
-- These are equivalent to: snow sql -q "SELECT * FROM tool_test"

-- Profile a query (equivalent to Snowsight Query Profile)
SELECT
    QUERY_ID,
    LEFT(QUERY_TEXT, 80) AS query_preview,
    EXECUTION_STATUS,
    TOTAL_ELAPSED_TIME / 1000 AS seconds,
    BYTES_SCANNED,
    PARTITIONS_SCANNED,
    PARTITIONS_TOTAL
FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
WHERE START_TIME >= DATEADD('minute', -30, CURRENT_TIMESTAMP())
  AND QUERY_TYPE NOT IN ('SHOW', 'DESCRIBE')
ORDER BY START_TIME DESC
LIMIT 5;

-- ============================================================
-- PART 5: Show parameters (accessible from any interface)
-- ============================================================
SHOW PARAMETERS IN SESSION;   -- Current session settings
SHOW PARAMETERS IN ACCOUNT;   -- Account-wide defaults
SHOW PARAMETERS IN WAREHOUSE learn_wh;  -- Warehouse-specific

-- ============================================================
-- PART 6: Snowsight Dashboard hint — create a named query
-- ============================================================
-- In Snowsight, you can save this as a tile on a Dashboard:
SELECT
    DATE_TRUNC('month', O_ORDERDATE) AS order_month,
    COUNT(*)                         AS orders,
```

```
    SUM(O_TOTALPRICE)                AS revenue
FROM SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.ORDERS
GROUP BY 1
ORDER BY 1;
-- After running: click the chart icon in the bottom of results
-- Choose "Bar chart", X = ORDER_MONTH, Y = REVENUE
-- Click "Save to Dashboard" to create a visualization

-- Clean up
DROP DATABASE IF EXISTS INTERFACE_DEMO;
```

Topic 1.2 — 20 Practice Questions

Q1. What is Snowsight?

- A) A command-line tool for running Snowflake scripts from a terminal
- B) Snowflake's primary browser-based web interface — accessed at app.snowflake.com
- C) A Python library for connecting to Snowflake from Jupyter Notebooks
- D) A dedicated VS Code extension only available to Enterprise customers

Q2. A data engineer wants to upload a local CSV file to a Snowflake stage using a terminal command in an automated shell script. Which tool is BEST suited?

- A) Snowsight — drag and drop the file in the browser
- B) Snowflake CLI — `snow stage put ./file.csv @stage`
- C) Snowflake ODBC driver — for file operations only
- D) VS Code Snowflake extension — run PUT commands from the editor

Q3. What is the "Query Profile" feature in Snowsight used for?

- A) Displaying the user's account profile and preferences
- B) A visual execution plan showing partitions scanned, bytes from cache, spill to disk — used for query performance analysis
- C) A saved template for frequently run queries
- D) An audit log of all queries run by a specific user

Q4. The Snowflake CLI command `snow sql -q "SELECT CURRENT_USER()"` does what?

- A) Opens Snowsight in the browser and runs the SQL
- B) Executes the SQL query from the terminal and returns the result to stdout
- C) Saves the SQL as a named query in the Snowflake account
- D) Creates a new worksheet in Snowsight

Q5. Which Snowflake interface is DEPRECATED (no longer the recommended primary interface)?

- A) Snowsight
- B) Snowflake CLI
- C) Classic Console (old web UI)
- D) VS Code Snowflake extension

Q6. A data analyst wants to build a visual bar chart dashboard showing monthly revenue without writing any charting code. Which Snowsight feature enables this?

- A) Snowflake Notebooks
- B) Streamlit in Snowflake
- C) Snowsight Dashboards — combine chart tiles built from SQL results
- D) Snowflake Marketplace

Q7. Which of the following tasks can be performed using the Snowflake CLI? *(Select ALL that apply)*

- A) Execute SQL queries from the terminal
- B) Deploy a Streamlit app to Snowflake
- C) Upload files to an internal stage
- D) Run Cortex AI summarization on a local text file
- E) Replace Snowflake's Cloud Services layer

Q8. Where is the Snowflake CLI configuration file stored by default?

- A) `C:\Program Files\Snowflake\config.ini`
- B) `~/.snowflake/config.toml`
- C) `/etc/snowflake/credentials.yaml`
- D) Inside the Snowflake account metadata store

Q9. A developer uses VS Code with the Snowflake extension. What does this integration allow? *(Select ALL that apply)*

- A) Browse Snowflake databases and schemas in a VS Code side panel
- B) Run SQL directly from VS Code with syntax highlighting and autocomplete
- C) Replace the need for any Virtual Warehouse
- D) Connect to Snowflake using account credentials from within VS Code

Q10. In Snowsight's Query History, which of the following information is available for a completed query?

- A) The physical file names of micro-partitions that were scanned
- B) Total elapsed time, bytes scanned, partitions scanned vs total, warehouse name, execution status

- C) The source code of the Snowflake optimization algorithm used
- D) The IP addresses of all warehouse nodes that processed the query

Q11. What replaced SnowSQL as the recommended Snowflake command-line tool?

- A) Snowsight terminal emulator
- B) Snowflake CLI (`snow` command)
- C) ODBC CLI connector
- D) Snowflake REST API client

Q12. You want to run a recurring SQL transformation script automatically in a CI/CD pipeline (e.g., GitHub Actions). Which tool is MOST appropriate?

- A) Snowsight — it has a built-in scheduler for worksheets
- B) Snowflake CLI — run SQL via `snow sql` in your CI/CD scripts
- C) VS Code extension — it supports GitHub Actions integration
- D) Snowflake Marketplace — pipeline templates are available there

Q13. Snowsight Notebooks support which programming languages? *(Select ALL that apply)*

- A) SQL
- B) Python
- C) R
- D) Java

Q14. Which Snowsight section allows an ACCOUNTADMIN to view credit consumption, create users, and manage role assignments?

- A) Activity tab
- B) Data tab
- C) Admin tab
- D) Marketplace tab

Q15. A business analyst shares a Snowsight worksheet with a colleague. What does the colleague receive access to?

- A) All databases and schemas in the account automatically
- B) The saved SQL query and its last result — they can run it with their own context (role, warehouse, database)
- C) A read-only view of the analyst's entire query history
- D) Permanent access to the analyst's current role and privileges

Q16. What is the `config.toml` file in Snowflake CLI used for?

- A) Storing encrypted table data for local development
- B) Defining named connection profiles (account URL, username, authentication method) for different environments
- C) Configuring Snowflake's internal query optimizer settings
- D) Storing SQL query templates for `snow sql` execution

Q17. The `snow cortex summarize --file report.txt` command in Snowflake CLI does what?

- A) Summarizes the SQL execution plan of a query
- B) Sends the contents of a local text file to Snowflake Cortex AI for summarization and returns the result
- C) Generates a Snowsight dashboard from a CSV file
- D) Creates a summary of the account's credit usage

Q18. Which interface would a data scientist use to run Python code interactively, visualize data inline, and train machine learning models — all within Snowflake?

- A) Snowflake CLI
- B) VS Code with ODBC connection
- C) Snowflake Notebooks in Snowsight (interactive Python/SQL cells)
- D) SnowSQL command-line client

Q19. In Snowsight, what does the "Data" tab allow you to do WITHOUT writing SQL?

- A) Edit data in Snowflake tables using a spreadsheet-like grid
- B) Browse the object hierarchy — databases, schemas, tables, columns — and see column data types and sample data
- C) View the execution plan for any query in the account
- D) Configure network policies and IP whitelists

Q20. A team wants to connect their existing BI tool (e.g., Tableau, Power BI) to Snowflake. Which Snowflake connectivity option enables this?

- A) Snowsight must be used for all BI tool connections
- B) Snowflake provides ODBC and JDBC drivers for connecting BI tools, plus native connectors for major platforms
- C) Only Enterprise edition supports BI tool connections
- D) BI tools must use the Snowflake CLI to query data

SUBDOMAIN 1.3 — Snowflake Object Hierarchy and Types

Layman's Explanation

Snowflake organizes everything in a neat hierarchy — like a Russian nesting doll:

Organization (outermost doll) = The entire corporation. Can contain multiple Snowflake accounts.
Account = One business unit / one Snowflake deployment. Your URL identifies it.
Database = Filing rooms. Contains all data for one logical domain.
Schema = Drawers in the filing room. Organizes tables and views by purpose (raw, staging, reporting).
Table, View, Stage, etc. = The actual documents and folders. The data lives here.

At the Account level: users, roles, warehouses, and resource monitors live here — shared across ALL databases.

At the Schema level: tables, views, stages, pipes, streams, tasks, sequences, UDFs, stored procedures — all live inside a specific schema.

The rule: **You must always know where you are.** Use `USE DATABASE` and `USE SCHEMA` to navigate. Reference objects by full three-part name: `DATABASE.SCHEMA.OBJECT`.

Technical Explanation

Full Object Hierarchy[8][1]

```
Organization
  └── Account(s)
        ├── ACCOUNT-LEVEL OBJECTS (shared across all databases)
        │     ├── Users              — login identities
        │     ├── Roles              — account roles (privilege containers)
        │     ├── Virtual Warehouses  — compute engines
        │     ├── Resource Monitors   — credit usage limits
        │     ├── Network Policies    — IP-based access control
        │     ├── Storage Integrations — IAM connections to external storage
        │     ├── API Integrations    — external API connections
        │     ├── Notification Integrations
        │     ├── Security Integrations (OAuth, SAML)
        │     ├── Shares             — data sharing objects
        │     └── Replication Groups  — cross-account replication
        │
        └── Database(s)
              ├── Database Roles     — scoped to this database only
              └── Schema(s)
                    └── SCHEMA-LEVEL OBJECTS:
                          ├── Tables (Permanent, Transient, Temp, Iceberg, External,
                          ├── Views (Standard, Materialized, Secure)
```

```
                    ├── External Tables
                    ├── Stages (Named Internal / Named External)
                    ├── File Formats
                    ├── Pipes (Snowpipe)
                    ├── Streams (CDC tracking)
                    ├── Tasks (scheduled SQL)
                    ├── Sequences (auto-increment generators)
                    ├── UDFs (User-Defined Functions)
                    ├── UDTFs (Table Functions)
                    ├── Stored Procedures
                    ├── Dynamic Tables
                    ├── ML Models (Snowflake ML)
                    ├── Snowflake Applications
                    └── Event Tables (for logging/tracing)
```

Auto-Created Schemas in Every Database

Every new database automatically gets two schemas:[^1]

1. PUBLIC — default schema; blank, ready for your objects
2. INFORMATION_SCHEMA — read-only, real-time metadata about every object in this database

Stage Types

Stages are named locations where files are stored before loading into tables or after unloading from tables.

Stage Type	Description	Syntax
User Stage	Auto-created for every user (`@~`). One per user.	`@~`
Table Stage	Auto-created for every table (`@%table_name`). One per table.	`@%my_table`
Named Internal Stage	Manually created stage inside Snowflake storage.	`CREATE STAGE my_stage`
Named External Stage	Manually created pointer to S3/Blob/GCS.	`CREATE STAGE my_ext STORAGE_INTEGRATION=...`

Edge cases on Stages:

- User and Table stages **cannot be dropped** — they are auto-managed
- Named stages **can** be dropped
- Named External stages require a **Storage Integration** (account-level object)
- Directory tables on a stage provide a `DIRECTORY(stage_name)` view of files in the stage

UDFs vs Stored Procedures

Feature	UDF	Stored Procedure
Returns	A single scalar value OR table (UDTF)	Can return value or nothing; executes procedural logic
Calling syntax	Used IN a SQL statement: SELECT `my_udf(col)`	Called WITH CALL: `CALL my_proc()`
Languages supported	JavaScript, Python, Java, Scala, SQL	JavaScript, Python, Java, Scala, Snowflake Scripting
Can run DML (INSERT/UPDATE)?	No	Yes
Snowpark library required for Python?	No (but available)	Yes (for Python/Java/Scala procs)

Sequences

- Auto-generate unique sequential integer values
- Created with `CREATE SEQUENCE seq_name START WITH 1 INCREMENT BY 1`
- Access with `seq_name.NEXTVAL` (returns next value) and `seq_name.CURRVAL` (current session value)
- **Edge case:** Sequences are NOT strictly sequential in all concurrent scenarios — gaps can occur due to caching. Use AUTOINCREMENT/IDENTITY on a column for guaranteed sequential integers within one table.

Parameter Hierarchy and Precedence

Settings (parameters) can be applied at multiple levels. More specific = higher priority:

```
Object Level  ← HIGHEST PRIORITY
   ↑
Session Level
   ↑
User Level
   ↑
Account Level ← LOWEST PRIORITY (broadest default)
```

Example: Account sets `DATA_RETENTION_TIME_IN_DAYS = 1`. An Enterprise table sets it to 30. The table uses 30 — object level wins.[^1]

Account Roles vs. Database Roles

Feature	Account Role	Database Role
Scope	Entire account	One specific database only
Assignable to users?	Yes, directly	No — must be granted to an account role first

Feature	Account Role	Database Role
Use in data sharing?	No	Yes — database roles can be granted to consumer accounts via shares
Created by	USERADMIN or SECURITYADMIN	SYSADMIN (or owner of database)

Session and Context Variables

Session variables (user-defined): Set with `SET myvar = value`. Reference with `$myvar`. Scoped to the current session only.

Context functions (system-defined): Always return current session context:

- `CURRENT_USER()`, `CURRENT_ROLE()`, `CURRENT_DATABASE()`, `CURRENT_SCHEMA()`, `CURRENT_WAREHOUSE()`, `CURRENT_TIMESTAMP()`, `CURRENT_REGION()`, `CURRENT_ACCOUNT()`

Hands-On Queries: Full Object Hierarchy

```
-- ============================================================
-- PART 1: Navigate the hierarchy
-- ============================================================
USE ROLE SYSADMIN;

-- Where am I?
SELECT
    CURRENT_USER()      AS user,
    CURRENT_ROLE()      AS role,
    CURRENT_DATABASE()  AS db,
    CURRENT_SCHEMA()    AS schema,
    CURRENT_WAREHOUSE() AS warehouse;

-- Create a full hierarchy
CREATE DATABASE IF NOT EXISTS HIERARCHY_DEMO
    COMMENT = 'Object hierarchy practice for COF-C03';

USE DATABASE HIERARCHY_DEMO;

-- Auto-created schemas:
SHOW SCHEMAS;  -- You'll see PUBLIC and INFORMATION_SCHEMA

-- Create custom schemas for Bronze/Silver/Gold architecture
CREATE SCHEMA IF NOT EXISTS BRONZE  COMMENT = 'Raw ingested data';
CREATE SCHEMA IF NOT EXISTS SILVER  COMMENT = 'Cleaned and validated data';
CREATE SCHEMA IF NOT EXISTS GOLD    COMMENT = 'Business-ready aggregations';

SHOW SCHEMAS;  -- Now: BRONZE, SILVER, GOLD, PUBLIC, INFORMATION_SCHEMA

-- ============================================================
-- PART 2: Create schema-level objects
-- ============================================================
USE SCHEMA BRONZE;
```

```
USE WAREHOUSE learn_wh;

-- Permanent Table
CREATE OR REPLACE TABLE raw_sales (
    sale_id      INTEGER,
    customer_id  INTEGER,
    product_code VARCHAR(20),
    quantity     INTEGER,
    unit_price   NUMBER(10,2),
    sale_date    DATE,
    region       VARCHAR(30),
    payload      VARIANT    -- for semi-structured data
);

-- Transient Table (no Fail-Safe, cheaper storage)
CREATE OR REPLACE TRANSIENT TABLE stg_sales_temp (
    sale_id      INTEGER,
    cleaned_flag BOOLEAN,
    load_ts      TIMESTAMP DEFAULT CURRENT_TIMESTAMP()
);

-- Temporary Table (session-scoped)
CREATE TEMPORARY TABLE session_work (
    id       INTEGER,
    result   NUMBER(12,2)
);

-- Sequence for auto-incrementing IDs
CREATE OR REPLACE SEQUENCE sales_seq
    START WITH 1000
    INCREMENT BY 1
    COMMENT = 'Sales ID generator';

-- Insert using sequence
INSERT INTO raw_sales
    (sale_id, customer_id, product_code, quantity, unit_price, sale_date, region)
VALUES
    (sales_seq.NEXTVAL, 101, 'PROD-A', 5, 29.99, '2024-01-15', 'North'),
    (sales_seq.NEXTVAL, 102, 'PROD-B', 2, 149.99,'2024-01-16', 'South'),
    (sales_seq.NEXTVAL, 103, 'PROD-A', 8, 29.99, '2024-01-17', 'East'),
    (sales_seq.NEXTVAL, 101, 'PROD-C', 1, 999.00,'2024-01-18', 'North'),
    (sales_seq.NEXTVAL, 104, 'PROD-B', 3, 149.99,'2024-01-19', 'West');

SELECT * FROM raw_sales ORDER BY sale_id;

-- Full 3-part name reference:
SELECT * FROM HIERARCHY_DEMO.BRONZE.RAW_SALES;

-- ============================================================
-- PART 3: Create views in SILVER schema
-- ============================================================
USE SCHEMA SILVER;

-- Standard View (always current, no storage cost)
CREATE OR REPLACE VIEW v_sales_clean AS
SELECT
```

```
    sale_id,
    customer_id,
    product_code,
    quantity,
    unit_price,
    quantity * unit_price AS line_total,
    sale_date,
    region,
    CASE
        WHEN region IN ('North','South') THEN 'Domestic'
        ELSE 'International'
    END AS region_group
FROM HIERARCHY_DEMO.BRONZE.RAW_SALES
WHERE quantity > 0 AND unit_price > 0;

SELECT * FROM v_sales_clean ORDER BY sale_id;

-- Secure View (hides SQL definition from unauthorized users)
CREATE OR REPLACE SECURE VIEW v_sales_public AS
SELECT
    sale_id,
    product_code,
    region,
    sale_date
FROM HIERARCHY_DEMO.BRONZE.RAW_SALES;
-- Non-owners cannot see the underlying SELECT logic

-- ============================================================
-- PART 4: Create GOLD aggregation view
-- ============================================================
USE SCHEMA GOLD;

CREATE OR REPLACE VIEW v_daily_revenue AS
SELECT
    sale_date,
    region,
    COUNT(*)              AS transactions,
    SUM(line_total)       AS revenue,
    AVG(line_total)       AS avg_order
FROM HIERARCHY_DEMO.SILVER.V_SALES_CLEAN
GROUP BY sale_date, region
ORDER BY sale_date, region;

SELECT * FROM v_daily_revenue;

-- ============================================================
-- PART 5: UDF creation
-- ============================================================
USE SCHEMA SILVER;

-- Simple SQL UDF
CREATE OR REPLACE FUNCTION classify_order(amount NUMBER)
RETURNS VARCHAR
AS $$
    CASE
        WHEN amount < 50    THEN 'Small'
```

```
        WHEN amount < 200   THEN 'Medium'
        WHEN amount < 1000  THEN 'Large'
        ELSE 'Enterprise'
    END
$$;

-- Use the UDF in a query
SELECT
    sale_id,
    quantity * unit_price AS line_total,
    classify_order(quantity * unit_price) AS order_size
FROM HIERARCHY_DEMO.BRONZE.RAW_SALES
ORDER BY sale_id;

-- JavaScript UDF example
CREATE OR REPLACE FUNCTION format_currency(amount FLOAT, symbol VARCHAR)
RETURNS VARCHAR
LANGUAGE JAVASCRIPT
AS $$
    return SYMBOL + AMOUNT.toFixed(2);
$$;

SELECT format_currency(299.50, '$') AS formatted;

-- ============================================================
-- PART 6: Parameter Hierarchy in action
-- ============================================================
USE ROLE ACCOUNTADMIN;

-- Check account-level retention default
SHOW PARAMETERS LIKE 'DATA_RETENTION%' IN ACCOUNT;

-- Set an account-level default
ALTER ACCOUNT SET DATA_RETENTION_TIME_IN_DAYS = 1;  -- Standard edition max

USE ROLE SYSADMIN;
-- Override at SESSION level
ALTER SESSION SET STATEMENT_TIMEOUT_IN_SECONDS = 300;
SHOW PARAMETERS LIKE 'STATEMENT_TIMEOUT%' IN SESSION;
-- Shows: 300 (session overrides account)

-- Reset session override
ALTER SESSION UNSET STATEMENT_TIMEOUT_IN_SECONDS;
SHOW PARAMETERS LIKE 'STATEMENT_TIMEOUT%' IN SESSION;
-- Now shows account-level value

-- ============================================================
-- PART 7: Session variables vs Context functions
-- ============================================================

-- User-defined session variable
SET reporting_date = '2024-01-17';
SET revenue_threshold = 500;

SELECT * FROM HIERARCHY_DEMO.BRONZE.RAW_SALES
WHERE sale_date = $reporting_date
```

```
  AND unit_price * quantity > $revenue_threshold;

-- Context variables (system-defined, always current)
SELECT
    CURRENT_USER()     AS who_am_i,
    CURRENT_ROLE()     AS my_role,
    $reporting_date    AS session_var_demo;

-- ============================================================
-- PART 8: INFORMATION_SCHEMA — real-time metadata
-- ============================================================
USE DATABASE HIERARCHY_DEMO;

SELECT TABLE_SCHEMA, TABLE_NAME, TABLE_TYPE, ROW_COUNT, BYTES
FROM INFORMATION_SCHEMA.TABLES
WHERE TABLE_SCHEMA NOT IN ('INFORMATION_SCHEMA')
ORDER BY TABLE_SCHEMA, TABLE_NAME;

SELECT TABLE_SCHEMA, TABLE_NAME, COLUMN_NAME, DATA_TYPE, IS_NULLABLE
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_SCHEMA = 'BRONZE'
  AND TABLE_NAME = 'RAW_SALES'
ORDER BY ORDINAL_POSITION;

SELECT TABLE_SCHEMA, TABLE_NAME, VIEW_DEFINITION
FROM INFORMATION_SCHEMA.VIEWS
WHERE TABLE_SCHEMA = 'SILVER';
-- Note: Secure View definition appears as NULL to non-owners!

-- Clean up
DROP DATABASE IF EXISTS HIERARCHY_DEMO;
```

Topic 1.3 — 20 Practice Questions

Q1. What is the correct order of Snowflake's object hierarchy from BROADEST to MOST SPECIFIC?

- A) Schema → Database → Account → Organization
- B) Organization → Account → Database → Schema → Object
- C) Account → Organization → Database → Schema → Table
- D) Database → Schema → Account → Organization → Table

Q2. Which of the following are ACCOUNT-LEVEL objects (not inside any database)? *(Select ALL that apply)*

- A) Virtual Warehouses
- B) Users
- C) Tables
- D) Resource Monitors

- E) Sequences
- F) Network Policies

Q3. What two schemas are automatically created in EVERY new Snowflake database?

- A) `ADMIN` and `SYSTEM`
- B) `DEFAULT` and `BACKUP`
- C) `PUBLIC` and `INFORMATION_SCHEMA`
- D) `STAGING` and `PRODUCTION`

Q4. What is the three-part (fully qualified) name for a table called `ORDERS` in schema `SALES` of database `COMPANY_DB`?

- A) `SALES.COMPANY_DB.ORDERS`
- B) `ORDERS.SALES.COMPANY_DB`
- C) `COMPANY_DB.SALES.ORDERS`
- D) `COMPANY_DB.ORDERS.SALES`

Q5. A user stage in Snowflake (`@~`) is:

- A) Created manually by ACCOUNTADMIN and assigned to specific users
- B) Automatically created for each user — one stage per user — and cannot be dropped
- C) A shared stage accessible by all users in the account
- D) Available only on Enterprise edition and above

Q6. What is the PRIMARY difference between a UDF (User-Defined Function) and a Stored Procedure in Snowflake?

- A) UDFs can be written in Python; Stored Procedures cannot
- B) UDFs are called within SQL statements and return scalar/table values; Stored Procedures are called with CALL and can execute DML (INSERT, UPDATE, DELETE)
- C) Stored Procedures are faster than UDFs for analytical workloads
- D) UDFs can only be written in JavaScript; Stored Procedures support all languages

Q7. Parameter precedence in Snowflake — which level has the HIGHEST priority?

- A) Account level
- B) User level
- C) Session level
- D) Object level

Q8. What does `INFORMATION_SCHEMA` contain in a Snowflake database?

- A) Audit logs of all user activity for the past year

- B) Real-time metadata about all objects in the current database — tables, columns, views, functions, stages
- C) Encrypted backup copies of all tables in the database
- D) Performance benchmarks comparing different query execution plans

Q9. Which stage type requires a Storage Integration account-level object to function?

- A) User stage (`@~`)
- B) Table stage (`@%table`)
- C) Named Internal stage
- D) Named External stage (pointing to S3/Azure Blob/GCS)

Q10. What is a Sequence's `NEXTVAL` used for?

- A) Getting the current timestamp as an integer
- B) Obtaining the next unique sequential integer value from the sequence for use as a primary key or ID
- C) Checking the last value returned across all sessions
- D) Resetting the sequence to its starting value

Q11. A Database Role differs from an Account Role in that:

- A) Database Roles can be granted directly to users; Account Roles cannot
- B) Database Roles are scoped to a specific database — they can be granted to consumer accounts for data sharing; Account Roles span the whole account
- C) Database Roles are created automatically; Account Roles must be created manually
- D) Database Roles can own warehouses; Account Roles cannot

Q12. An account-level `DATA_RETENTION_TIME = 7` is set. A specific table overrides it with `DATA_RETENTION_TIME = 30`. Which retention applies to that table?

- A) 7 days — account-level always wins
- B) 30 days — object-level (the table's setting) has higher precedence than account-level
- C) Average of both: 18.5 days
- D) 30 days only if the user has ACCOUNTADMIN role

Q13. What is the purpose of a File Format object in Snowflake?

- A) It defines the schema of a table during CREATE TABLE
- B) It stores a named definition of an external file structure (CSV delimiter, JSON settings, Parquet schema) — reused across COPY INTO and external table operations
- C) It converts Parquet files into Snowflake's internal format automatically
- D) It is used to format query results before displaying in Snowsight

Q14. True or False: You can drop a User Stage (`@~`) or Table Stage (`@%table`).

- A) True — any stage can be dropped with DROP STAGE
- B) False — User and Table stages are auto-managed and cannot be dropped
- C) True — but only ACCOUNTADMIN can drop auto-created stages
- D) False — only Named Internal stages can be dropped; Named External stages cannot

Q15. A company stores marketing data, HR data, and finance data in one Snowflake account. What is the RECOMMENDED way to organize these at the top container level within the account?

- A) One schema per domain (marketing, HR, finance) inside a single database
- B) One separate database per domain, each with their own schemas
- C) One warehouse per domain with separate data loaded into each warehouse
- D) One account per domain — accounts are the recommended organizational unit

Q16. Which programming languages are supported for writing UDFs in Snowflake? *(Select ALL that apply)*

- A) JavaScript
- B) Python
- C) Java
- D) Scala
- E) SQL
- F) R

Q17. What does `SET my_date = '2024-06-15'` followed by `SELECT * FROM t WHERE sale_date = $my_date` demonstrate?

- A) A permanent table variable stored in the INFORMATION_SCHEMA
- B) A session-scoped user-defined variable used in a query — the `$` prefix references the variable
- C) A Snowflake stored procedure calling convention
- D) An account-level parameter override

Q18. A schema is accidentally dropped. What command recovers it and all its tables?

- A) `RESTORE SCHEMA schema_name`
- B) `UNDROP SCHEMA schema_name` — restores the schema and all objects within it
- C) `ROLLBACK SCHEMA schema_name`
- D) `SELECT * FROM schema_name AT (OFFSET => -60)` to re-create from history

Q19. Which object type provides a view of files stored in a stage and allows querying file metadata?

- A) Pipe
- B) Stream
- C) Directory Table (enabled on a named stage)
- D) File Format

Q20. `CURRVAL` on a Sequence returns:

- A) The highest value ever produced by the sequence across all sessions
- B) The most recent value returned by `NEXTVAL` in the CURRENT session
- C) The starting value the sequence was created with
- D) The current maximum value in the target table's ID column

SUBDOMAIN 1.4 — Virtual Warehouses Configuration

Layman's Explanation

Virtual Warehouses are Snowflake's compute engines — the workers that do the actual query processing. Understanding them deeply is one of the most-tested areas of Domain 1. [^1]

Think of warehouse sizing like hiring a construction crew:

- **X-Small (1 worker)**: Fine for a small job. Slow for a big job.
- **Large (8 workers)**: 8× faster on a big job, 8× more expensive per hour.
- **Scale Up** (bigger size) = hire more skilled workers for ONE complex job.
- **Scale Out** (multi-cluster) = open MORE identical job sites to serve MORE customers simultaneously.

The billing key: you pay for the **time the crew is at the site**, not just when they're actively swinging hammers. Show up for 10 seconds? You pay for 60 seconds minimum. But if you send them home during lunch (auto-suspend), you pay nothing while they're gone.

Technical Explanation

Warehouse Types[5][8][^1]

Type	Description	Use Cases
Standard (Gen 2)	Current default. Balanced CPU/memory.	SQL analytics, ETL, general workloads
Standard (Gen 1)	Older generation. Still available.	Legacy workloads already using Gen 1
Snowpark-Optimized	16× more memory per node.	Python UDFs, ML model training/inference, large in-memory processing

Gen 1 vs Gen 2 key differences:

- Gen 2 is the default for all new warehouses created today
- Gen 2 provides better CPU/memory ratios than Gen 1
- Specify `WAREHOUSE_TYPE = 'STANDARD'` for Gen 2 (default) or use specific Gen 1 option if needed

Warehouse Sizes and Credits

Size	Credits/Hour	Relative Power
X-Small	1	Baseline
Small	2	2× XS
Medium	4	4× XS
Large	8	8× XS
X-Large	16	16× XS
2X-Large	32	32× XS
3X-Large	64	64× XS
4X-Large	128	128× XS

Each size doubles both credits AND compute nodes.[^1]

Billing Rules (Memorize All)

1. Billing is **per second** (not per minute or per hour)
2. **Minimum 60-second charge per start event** (i.e., every time state goes SUSPENDED → STARTED)
3. Billing starts when warehouse transitions from `SUSPENDED` → `STARTED`
4. Billing stops the moment state transitions back to `SUSPENDED`
5. Resizing while active: existing queries run at old size; new queries use new size
6. **Each start of a warehouse = new 60-second minimum**, even if the previous run was only 30 seconds

Auto-Suspend and Auto-Resume

Setting	Behavior
`AUTO_SUSPEND = N`	After N seconds of zero query activity, warehouse automatically suspends. Billing stops.
`AUTO_RESUME = TRUE` (default)	When a query is submitted to a suspended warehouse, it automatically resumes (and a new 60-second minimum begins).
`AUTO_RESUME = FALSE`	Queries submitted to a suspended warehouse fail immediately — must be manually resumed.

Edge case on AUTO_SUSPEND: The timer resets whenever any query begins execution. A warehouse won't suspend while a query is running — only after the configured idle seconds of NO query activity.

Multi-Cluster Warehouses (Enterprise Edition+)[^1]

Used when ONE warehouse cluster can't serve all concurrent users without queuing.

Parameter	Behavior
`MIN_CLUSTER_COUNT`	Always-running clusters (never go below this even at 0 queries)
`MAX_CLUSTER_COUNT`	Maximum clusters allowed to spin up under load
`SCALING_POLICY = 'STANDARD'`	Start a new cluster the moment ANY query is queued — minimize wait, use more credits
`SCALING_POLICY = 'ECONOMY'`	Wait for consistent queuing before starting new clusters — save credits, accept some queue time

Scale Up vs. Scale Out — Critical Distinction:

Scenario	Solution	Why
One complex 10-minute query runs slowly	Scale Up — larger warehouse	More CPUs/RAM per node speeds single query
200 users queuing for 1-second queries	Scale Out — multi-cluster	Each cluster handles a separate user queue

Warehouse Configuration by Use Case[^1]

Use Case	Recommended Config
Developer/testing	X-Small, AUTO_SUSPEND=60, AUTO_RESUME=TRUE
BI dashboards (high concurrency)	Multi-cluster, STANDARD scaling, Medium+ size
Overnight ETL batch	Large/X-Large, AUTO_SUSPEND=300, single cluster
ML model training (Snowpark Python)	Snowpark-Optimized, Large+
Ad-hoc analyst queries	Small–Medium, AUTO_SUSPEND=120

❄️ Hands-On Queries: Virtual Warehouse Mastery

```
-- ============================================================
-- PART 1: Create warehouses for different use cases
-- ============================================================
USE ROLE SYSADMIN;

-- Developer warehouse: smallest, quick suspend
CREATE OR REPLACE WAREHOUSE wh_dev
    WAREHOUSE_SIZE   = 'X-SMALL'
    AUTO_SUSPEND     = 60
    AUTO_RESUME      = TRUE
    COMMENT          = 'For developers — auto-suspends in 60s';

-- Production BI warehouse: bigger, slower suspend
CREATE OR REPLACE WAREHOUSE wh_bi
    WAREHOUSE_SIZE   = 'MEDIUM'
    AUTO_SUSPEND     = 300
    AUTO_RESUME      = TRUE
    COMMENT          = 'For BI analysts — 5 min idle before suspend';

-- ETL batch warehouse
CREATE OR REPLACE WAREHOUSE wh_etl
    WAREHOUSE_SIZE   = 'LARGE'
    AUTO_SUSPEND     = 600
    AUTO_RESUME      = TRUE
    COMMENT          = 'For heavy ETL jobs — 10 min idle';

-- Snowpark / ML warehouse (conceptual — shows the TYPE setting)
CREATE OR REPLACE WAREHOUSE wh_ml
    WAREHOUSE_SIZE   = 'MEDIUM'
    WAREHOUSE_TYPE   = 'SNOWPARK-OPTIMIZED'    -- KEY: 16x more memory!
    AUTO_SUSPEND     = 120
    AUTO_RESUME      = TRUE
    COMMENT          = 'For Python UDFs and ML model training';

SHOW WAREHOUSES;
-- Observe columns: size, state, warehouse_type, auto_suspend, credits_per_hour

-- ============================================================
-- PART 2: Billing behavior — the 60-second minimum
-- ============================================================
USE WAREHOUSE wh_dev;

-- Start a stopwatch in your head — this starts the 60-second minimum
SELECT CURRENT_TIMESTAMP() AS start_time;

-- This query takes 1 second but you're billed for at least 60 seconds
SELECT COUNT(*) FROM SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.ORDERS;

-- Manually suspend immediately
ALTER WAREHOUSE wh_dev SUSPEND;
-- You just used AT LEAST 60 seconds worth of credits (1 credit/hr × 60/3600 hr = 0.01
```

```
-- Resume — this starts a FRESH 60-second minimum billing cycle
ALTER WAREHOUSE wh_dev RESUME;

-- ============================================================
-- PART 3: AUTO_RESUME = FALSE behavior
-- ============================================================
CREATE OR REPLACE WAREHOUSE wh_manual
    WAREHOUSE_SIZE = 'X-SMALL'
    AUTO_SUSPEND   = 60
    AUTO_RESUME    = FALSE;    -- Must be manually resumed!

-- Suspend it
ALTER WAREHOUSE wh_manual SUSPEND;

-- Try a query on it (this will FAIL because AUTO_RESUME = FALSE)
USE WAREHOUSE wh_manual;
SELECT 1;
-- Error: "Warehouse WH_MANUAL cannot be resumed automatically..."

-- Manual resume fixes it
ALTER WAREHOUSE wh_manual RESUME;
SELECT 1;  -- Now works

DROP WAREHOUSE IF EXISTS wh_manual;

-- ============================================================
-- PART 4: Scaling — Up vs Out demonstration
-- ============================================================

-- SCALE UP: Resize from X-Small to Large for a heavy query
USE WAREHOUSE wh_dev;

ALTER WAREHOUSE wh_dev SET WAREHOUSE_SIZE = 'LARGE';
SHOW WAREHOUSES LIKE 'WH_DEV';  -- Verify new size

-- Run a heavy query (benefits from the larger warehouse)
SELECT
    L_RETURNFLAG,
    L_LINESTATUS,
    SUM(L_QUANTITY)        AS sum_qty,
    SUM(L_EXTENDEDPRICE)   AS sum_base_price,
    AVG(L_DISCOUNT)        AS avg_discount,
    COUNT(*)               AS count_order
FROM SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.LINEITEM
GROUP BY L_RETURNFLAG, L_LINESTATUS
ORDER BY L_RETURNFLAG, L_LINESTATUS;

-- Scale back down for regular use
ALTER WAREHOUSE wh_dev SET WAREHOUSE_SIZE = 'X-SMALL';

-- SCALE OUT (Enterprise edition concept):
-- CREATE OR REPLACE WAREHOUSE wh_multicluster
--     WAREHOUSE_SIZE     = 'MEDIUM'
--     MIN_CLUSTER_COUNT  = 1
--     MAX_CLUSTER_COUNT  = 4
--     SCALING_POLICY     = 'STANDARD';
```

```
-- (Requires Enterprise; this will error on Standard edition)

-- ============================================================
-- PART 5: Monitor credit consumption
-- ============================================================
USE ROLE ACCOUNTADMIN;

SELECT
    WAREHOUSE_NAME,
    ROUND(SUM(CREDITS_USED), 4)         AS total_credits,
    ROUND(SUM(CREDITS_USED) * 3.0, 2)  AS estimated_cost_usd,  -- ~$3/credit estimat
    COUNT(*)                            AS billing_periods,
    MIN(START_TIME)                     AS first_use,
    MAX(END_TIME)                       AS last_use
FROM SNOWFLAKE.ACCOUNT_USAGE.WAREHOUSE_METERING_HISTORY
WHERE START_TIME >= DATEADD('day', -1, CURRENT_TIMESTAMP())
GROUP BY WAREHOUSE_NAME
ORDER BY total_credits DESC;

-- ============================================================
-- PART 6: Credit cost calculations
-- ============================================================

-- Calculate cost for different warehouse sizes running different durations:
SELECT
    size,
    credits_per_hour,
    ROUND(credits_per_hour * (duration_minutes / 60), 4) AS credits_used
FROM (VALUES
    ('X-Small',  1,   15),   -- X-Small running 15 minutes
    ('Small',    2,   30),   -- Small running 30 minutes
    ('Medium',   4,   60),   -- Medium running 1 hour
    ('Large',    8,    7.5), -- Large running 7.5 minutes
    ('X-Large',  16,   2)    -- X-Large running 2 minutes
) AS t(size, credits_per_hour, duration_minutes);

-- Clean up
DROP WAREHOUSE IF EXISTS wh_dev;
DROP WAREHOUSE IF EXISTS wh_bi;
DROP WAREHOUSE IF EXISTS wh_etl;
DROP WAREHOUSE IF EXISTS wh_ml;
```

Topic 1.4 — 20 Practice Questions

Q1. A Medium warehouse (4 credits/hour) runs for exactly 45 minutes. How many credits are consumed?

- A) 4.0 credits
- B) 3.0 credits
- C) 2.0 credits
- D) 1.0 credits

Q2. What is the minimum billing duration every time a warehouse STARTS or RESUMES from SUSPENDED state?

- A) 10 seconds
- B) 30 seconds
- C) 60 seconds
- D) 300 seconds

Q3. A warehouse has `AUTO_RESUME = FALSE` and is currently SUSPENDED. A user submits a query to it. What happens?

- A) The warehouse automatically resumes and runs the query
- B) The query is queued until a SYSADMIN manually resumes the warehouse
- C) The query fails immediately with an error — AUTO_RESUME = FALSE means queries cannot auto-start the warehouse
- D) The query runs using a fallback X-Small warehouse

Q4. Which warehouse type provides 16× more memory per node, specifically designed for Snowpark Python and ML workloads?

- A) Standard Gen 2
- B) Standard Gen 1
- C) Snowpark-Optimized
- D) Enterprise-Performance

Q5. 200 business users all submit dashboard queries simultaneously. Queries begin queuing on a Medium warehouse. What is the CORRECT solution?

- A) Scale Up — resize to X-Large warehouse
- B) Scale Out — enable multi-cluster with a higher MAX_CLUSTER_COUNT
- C) Increase AUTO_SUSPEND time to keep the warehouse warm
- D) Create a second separate database to split user load

Q6. A single complex analytical query joining three 100 GB tables runs slowly. There is no user concurrency issue. What is the BEST fix?

- A) Scale Out — enable multi-cluster warehouses
- B) Scale Up — use a larger warehouse size for more CPU/RAM per query
- C) Enable AUTO_RESUME = FALSE to prioritize this query
- D) Create a clone of the tables to reduce contention

Q7. Multi-cluster warehouses require which MINIMUM Snowflake edition?

- A) Standard

- B) Enterprise
- C) Business Critical
- D) Virtual Private Snowflake

Q8. A Large warehouse (8 credits/hr) runs for exactly 7 minutes and 30 seconds. What is the exact credit cost?

- A) 8 credits (rounded up to 1 hour minimum)
- B) 1.0 credits
- C) 0.5 credits (7.5 minutes × 8/60 = 1 credit... wait let me calculate: 8 × (7.5/60) = 1.0)
- D) 4.0 credits

Q9. What does `SCALING_POLICY = 'ECONOMY'` do in a multi-cluster warehouse?

- A) Starts a new cluster the moment any single query is queued — minimizes wait time
- B) Waits for consistent, sustained queuing before starting additional clusters — saves credits but allows some queue time
- C) Reduces the size of each cluster to the minimum (X-Small) to save cost
- D) Limits the warehouse to 1 cluster maximum regardless of MAX_CLUSTER_COUNT

Q10. An X-Small warehouse runs for 10 seconds, then auto-suspends. It resumes again 5 minutes later and runs for 10 more seconds, then auto-suspends. What is the TOTAL credits billed?

- A) 0.006 credits (20 seconds of actual X-Small usage)
- B) 0.017 credits (60 seconds at 1 credit/hr)
- C) 0.033 credits (2 × 60-second minimums at 1 credit/hr)
- D) 0.5 credits (1 full credit charged per start event)

Q11. When you resize a warehouse from Small to Large while a query is actively running, what happens to the in-flight query?

- A) The query immediately gains the Large warehouse's resources and speeds up
- B) The query is cancelled and must be re-submitted on the Large warehouse
- C) The in-flight query completes at the old Small size; only new queries after the resize use Large
- D) Resizing during active query execution is not permitted

Q12. What does `MIN_CLUSTER_COUNT = 3` mean in a multi-cluster warehouse?

- A) The warehouse starts with 3 nodes but can scale down to 1 node during low traffic
- B) At least 3 clusters are always running — they never drop below 3, even with zero queries
- C) 3 queries must be queued before the first cluster starts

- D) The warehouse has a minimum of 3 concurrent users

Q13. Which of the following are TRUE about Virtual Warehouse billing? *(Select ALL that apply)*

- A) Billing is per second with a 60-second minimum per start event
- B) Billing stops the moment the warehouse transitions to SUSPENDED state
- C) A warehouse that has been running for 50 seconds is charged for 60 seconds when suspended
- D) Running multiple warehouses simultaneously triples the data storage cost
- E) Dropping a warehouse stops all future billing for that account

Q14. A developer wants a warehouse that automatically starts when they submit queries but aggressively shuts down after 1 minute of inactivity to minimize costs. What configuration achieves this?

- A) `AUTO_SUSPEND = 60, AUTO_RESUME = FALSE`
- B) `AUTO_SUSPEND = 60, AUTO_RESUME = TRUE`
- C) `AUTO_SUSPEND = 300, AUTO_RESUME = TRUE`
- D) `AUTO_SUSPEND = 0, AUTO_RESUME = TRUE`

Q15. Which context function shows the name of the currently active warehouse in your session?

- A) `SELECT ACTIVE_WAREHOUSE()`
- B) `SELECT CURRENT_WAREHOUSE()`
- C) `SELECT SESSION_WAREHOUSE()`
- D) `SELECT GET_WAREHOUSE()`

Q16. You create a warehouse with `WAREHOUSE_SIZE = 'X-SMALL'` and `WAREHOUSE_TYPE = 'SNOWPARK-OPTIMIZED'`. What is the primary difference compared to a Standard X-Small?

- A) Snowpark-Optimized is cheaper per credit
- B) Snowpark-Optimized has 16× more memory per node — ideal for Python UDFs and ML workloads
- C) Snowpark-Optimized is the only type that supports multi-cluster
- D) Snowpark-Optimized automatically resizes based on query complexity

Q17. A data engineer needs to run a 4-hour batch ETL job overnight. Which warehouse configuration is MOST appropriate?

- A) X-Small with AUTO_SUSPEND = 60 to minimize cost if it idles
- B) Large with AUTO_SUSPEND = 600 — larger warehouse finishes faster; longer suspend threshold avoids mid-job suspension
- C) Multi-cluster X-Small with 10 clusters — parallel ETL execution

- D) Snowpark-Optimized X-Small — best for all ETL regardless of workload type

Q18. What is the effect of running `ALTER WAREHOUSE wh_prod SUSPEND`?

- A) Permanently deletes the warehouse definition
- B) Immediately stops all running queries and billing
- C) Immediately stops billing; any currently running queries complete first, then the warehouse suspends
- D) Marks the warehouse for suspension after current queries complete, without stopping billing immediately

Q19. How does `STANDARD` scaling policy differ from `ECONOMY` in multi-cluster warehouses?

- A) STANDARD charges per query; ECONOMY charges per hour
- B) STANDARD adds a new cluster as soon as any query is queued; ECONOMY waits for consistent sustained load before adding clusters
- C) STANDARD uses Gen 2 nodes; ECONOMY uses Gen 1 nodes
- D) STANDARD allows Max 4 clusters; ECONOMY allows Max 10 clusters

Q20. Which of the following are TRUE about Snowflake Virtual Warehouses? *(Select ALL that apply)*

- A) A warehouse can be resized while queries are running
- B) Multiple warehouses can simultaneously query the same table with no storage conflicts
- C) Dropping a warehouse permanently deletes all tables associated with it
- D) A Snowpark-Optimized warehouse has more memory per node than a Standard warehouse of the same size
- E) Auto-Suspend suspends the warehouse only after all running queries complete

SUBDOMAIN 1.5 — Snowflake Storage Concepts

Layman's Explanation

This subdomain covers the "**what is actually stored and how**" question in Snowflake. Five key concepts:

1. **Micro-partitions**: How data is cut up and stored (labeled boxes in a warehouse)
2. **Data clustering**: How to keep related boxes together so you find them faster
3. **Table types**: Different categories of tables for different purposes
4. **Iceberg tables**: Tables whose data is stored in an open format you control
5. **View types**: "Windows" into your data — standard, materialized, and secure

The critical exam insight: Snowflake has **6 table types** and **3 view types**. You must know exactly which feature (Time Travel, Fail-Safe, write capability, storage location) applies to each type.

⚙ Technical Explanation

Micro-Partitions[3][4]

- **Size:** 50–500 MB uncompressed (typically ~100 MB compressed)
- **Format:** Columnar within each partition
- **Immutability:** Never modified in place — DML creates new partitions
- **Metadata per partition** (stored in Cloud Services): min value, max value, count, null count per column
- **Partition pruning:** Query optimizer uses column min/max metadata to skip irrelevant partitions entirely

Data Clustering[^1]

Natural clustering: Data inserts in order = naturally clustered (e.g., time-series data loaded daily clusters by date naturally).

Clustering keys: User-defined column(s) that tell Snowflake to co-locate rows with similar values in the same micro-partitions.

When to use clustering keys:

- Tables > 1 TB with consistent filter queries on specific columns
- Columns with high cardinality used in WHERE/JOIN filters
- NOT for small tables, low-cardinality columns, or ad-hoc queries

Automatic Clustering: Snowflake background service that continuously re-sorts micro-partitions to maintain clustering depth. Costs serverless credits (not warehouse credits).

Clustering depth: Average overlapping micro-partitions. Lower = better clustering = more pruning.

Table Types — COMPLETE Matrix[10][8][^1]

Feature	Permanent	Transient	Temporary	External	Dynamic	Iceberg
Time Travel	0–90 days*	0–1 day	0–1 day	None	None	Limited**
Fail-Safe	7 days	None	None	None	None	None
Session-only	No	No	Yes	No	No	No
Auto-dropped	No	No	On session end	No	No	No

Feature	Permanent	Transient	Temporary	External	Dynamic	Iceberg
Writable	Yes	Yes	Yes	No (read-only)	No (auto-managed)	Yes (Snowflake catalog)
Storage	Snowflake	Snowflake	Snowflake	External (S3/Blob/GCS)	Snowflake	External (Parquet files)
Fail-Safe billing	Yes	No	No	No	No	No

*Enterprise+ for >1 day
**Iceberg tables support Time Travel via snapshot-based queries

Apache Iceberg Tables[11][12][^10]

What they are: Tables that store data in the **open Apache Iceberg table format** (using Parquet files) in external cloud storage. This allows other tools (Spark, Flink, Trino) to read the same data directly.

Key features:

- ACID transactions, schema evolution, hidden partitioning
- Two catalog modes:
  - **Snowflake as catalog:** Full read/write DML support. Automatic clustering, compaction, zero-copy cloning supported. Uses Snowpipe Streaming.
  - **External catalog (Glue, Open Table Catalog):** **Read-only** in Snowflake — cannot INSERT/UPDATE/DELETE
- File format: **Parquet only** (not Avro, not ORC for Iceberg in Snowflake)
- Use case: Sharing data between Snowflake and other systems; data lake modernization

Critical exam traps for Iceberg:

- External catalog Iceberg tables = read-only in Snowflake
- Snowflake-catalog Iceberg tables = full read/write
- Time Travel on Iceberg uses `AT (SNAPSHOT => snapshot_id)` or `AT (TIME => ts)` — different from native tables
- Zero-copy cloning works on Snowflake-catalog Iceberg tables

View Types[13][14][^15]

Standard View (Non-Materialized)

- A saved SQL query — no physical data storage
- Always returns current data (executed dynamically every time)
- Full SQL flexibility: joins, subqueries, window functions, multiple tables
- Available on ALL editions
- Zero storage cost
- Performance: slower for complex queries (recomputed each time)

Materialized View

- Pre-computed, stored query results — physically stores data like a table
- **Automatically maintained** by Snowflake's serverless maintenance process (background cost!)
- Always current — updated as base table changes
- **Restrictions** (critical for exam):
  - **Single base table only** — NO JOINS between multiple tables
  - No aggregations with HAVING clauses
  - No window functions (OVER clause)
  - No LIMIT/ORDER BY in view definition
  - No GROUP BY on columns with high cardinality in some cases
- Requires **Enterprise edition** or higher
- Used for: Frequent identical subqueries, filter-heavy dashboards

Secure View

- Can be applied to BOTH standard AND materialized views
- Hides the view definition SQL from unauthorized users (`SHOW VIEWS` shows definition as NULL)
- Disables optimizer pushdown optimization (slightly slower but prevents inference attacks)
- Available on ALL editions (even Standard)
- Required for data sharing via Snowflake Shares (only secure views and secure materialized views can be shared)

Critical view exam traps:

Statement	True/False
Materialized views need Enterprise edition	TRUE
Standard views need Enterprise edition	FALSE — all editions

Statement	True/False
Secure views are only for materialized views	FALSE — can be applied to standard views too
A view can be both Secure AND Materialized	TRUE — CREATE SECURE MATERIALIZED VIEW
Materialized views support JOINs across tables	FALSE — single table only

Hands-On Queries: Storage Concepts

```
-- ============================================================
-- SETUP
-- ============================================================
USE ROLE SYSADMIN;
CREATE DATABASE IF NOT EXISTS STORAGE_DEMO;
USE DATABASE STORAGE_DEMO;
CREATE SCHEMA IF NOT EXISTS DEMO;
USE SCHEMA DEMO;
USE WAREHOUSE learn_wh;

-- ============================================================
-- PART 1: All six table types
-- ============================================================

-- 1. Permanent Table (default)
CREATE OR REPLACE TABLE perm_orders (
    order_id    INTEGER,
    customer_id INTEGER,
    amount      NUMBER(12,2),
    order_date  DATE,
    status      VARCHAR(20)
);

-- 2. Transient Table (no Fail-Safe)
CREATE OR REPLACE TRANSIENT TABLE stg_orders (
    order_id    INTEGER,
    raw_data    VARIANT,
    loaded_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP()
);

-- 3. Temporary Table (session-scoped)
CREATE TEMPORARY TABLE tmp_work (
    order_id INTEGER,
    rank_num INTEGER
);

-- 4. Dynamic Table (auto-refreshing)
-- First create a source table:
CREATE OR REPLACE TABLE raw_events (
    event_id   INTEGER,
    event_type VARCHAR,
    user_id    INTEGER,
    ts         TIMESTAMP
);
```

```
INSERT INTO raw_events VALUES
    (1,'login',   101, '2024-01-15 08:00:00'),
    (2,'buy',     101, '2024-01-15 08:05:00'),
    (3,'login',   102, '2024-01-15 08:10:00'),
    (4,'logout',  101, '2024-01-15 08:30:00');

CREATE OR REPLACE DYNAMIC TABLE user_summary
    TARGET_LAG = '1 minute'
    WAREHOUSE  = learn_wh
AS
SELECT
    user_id,
    COUNT(*)                                          AS total_events,
    COUNT(CASE WHEN event_type='buy' THEN 1 END)      AS purchases,
    MAX(ts)                                           AS last_seen
FROM raw_events
GROUP BY user_id;

SELECT * FROM user_summary ORDER BY user_id;

-- 5. Show all table types in SHOW TABLES
SHOW TABLES IN SCHEMA STORAGE_DEMO.DEMO;
-- Observe: kind=TABLE for all, is_transient flag for transient

-- ============================================================
-- PART 2: Time Travel comparison across table types
-- ============================================================

-- Permanent: supports up to 90 days (Enterprise) or 1 day (Standard)
ALTER TABLE perm_orders SET DATA_RETENTION_TIME = 1;  -- 1 day
SHOW PARAMETERS LIKE 'DATA_RETENTION%' IN TABLE perm_orders;

-- Transient: MAX 1 day, no Fail-Safe
ALTER TABLE stg_orders SET DATA_RETENTION_TIME = 1;

-- Try to set transient retention > 1 (will fail):
ALTER TABLE stg_orders SET DATA_RETENTION_TIME = 7;
-- Error: "Transient tables only support DATA_RETENTION_TIME of 0 or 1"

-- ============================================================
-- PART 3: Three view types
-- ============================================================

-- Insert data for views practice
INSERT INTO perm_orders VALUES
    (1, 101, 250.00,  '2024-01-15', 'ACTIVE'),
    (2, 102, 89.50,   '2024-01-16', 'ACTIVE'),
    (3, 103, 1500.00, '2024-01-17', 'VIP'),
    (4, 101, 45.00,   '2024-01-18', 'ACTIVE'),
    (5, 104, 300.00,  '2024-01-19', 'ACTIVE');

-- Standard View (no storage, always current)
CREATE OR REPLACE VIEW v_active_orders AS
SELECT
    order_id,
    customer_id,
```

```
    amount,
    order_date,
    CASE WHEN amount > 200 THEN 'Premium' ELSE 'Standard' END AS tier
FROM perm_orders
WHERE status = 'ACTIVE';

SELECT * FROM v_active_orders ORDER BY order_id;

-- Add a new row and re-query view — it shows the update IMMEDIATELY
INSERT INTO perm_orders VALUES (6, 105, 500.00, '2024-01-20', 'ACTIVE');
SELECT * FROM v_active_orders ORDER BY order_id;
-- New row visible immediately — standard view is always current!

-- Secure View (hides definition from non-owners)
CREATE OR REPLACE SECURE VIEW v_orders_public AS
SELECT
    order_id,
    order_date,
    CASE WHEN amount > 500 THEN 'High Value' ELSE 'Standard' END AS value_band
FROM perm_orders;

SELECT * FROM v_orders_public ORDER BY order_id;

-- Check: Non-owners see NULL for view definition
SHOW VIEWS LIKE 'V_ORDERS_PUBLIC';
-- is_secure = YES; definition shows NULL to non-owners

-- Materialized View (Enterprise only — will error on Standard)
CREATE OR REPLACE MATERIALIZED VIEW mv_customer_totals AS
SELECT
    customer_id,
    COUNT(*)      AS order_count,
    SUM(amount)   AS total_spent,
    MAX(amount)   AS max_order
FROM perm_orders
GROUP BY customer_id;
-- Standard edition: "Materialized View not supported in current edition"
-- Enterprise edition: Creates successfully and auto-refreshes!

DROP MATERIALIZED VIEW IF EXISTS mv_customer_totals;

-- ============================================================
-- PART 4: Micro-partition and clustering concepts
-- ============================================================

-- Create a large table to observe partition behavior
CREATE OR REPLACE TABLE sales_data (
    sale_id   INTEGER,
    sale_date DATE,
    region    VARCHAR(20),
    amount    NUMBER(12,2)
);

-- Generate data across 2 years
INSERT INTO sales_data
SELECT
```

```
    ROW_NUMBER() OVER (ORDER BY SEQ8())   AS sale_id,
    DATEADD('day',
        UNIFORM(0, 730, RANDOM()),
        '2023-01-01')::DATE              AS sale_date,
    CASE MOD(SEQ8(), 4)
        WHEN 0 THEN 'North'
        WHEN 1 THEN 'South'
        WHEN 2 THEN 'East'
        ELSE 'West'
    END                                  AS region,
    ROUND(UNIFORM(10.0, 5000.0, RANDOM()),2) AS amount
FROM TABLE(GENERATOR(ROWCOUNT => 500000));

-- Check table info
SELECT TABLE_NAME, ROW_COUNT, ROUND(BYTES/1048576,2) AS size_mb
FROM INFORMATION_SCHEMA.TABLES
WHERE TABLE_NAME = 'SALES_DATA';

-- Run a filtered query — observe partition pruning in Query Profile!
-- Click "Query Profile" after running:
SELECT SUM(amount)
FROM sales_data
WHERE sale_date = '2024-06-15';
-- In Query Profile: "Partitions scanned" should be << "Partitions total"

-- Add a clustering key on sale_date (natural filter column)
ALTER TABLE sales_data CLUSTER BY (sale_date);
SELECT SYSTEM$CLUSTERING_INFORMATION('sales_data', '(sale_date)');
-- After Automatic Clustering runs, average_depth should decrease

-- Iceberg table concept (read-only via external catalog):
-- CREATE ICEBERG TABLE iceberg_sales
--     EXTERNAL_VOLUME = 'my_s3_volume'
--     CATALOG = 'SNOWFLAKE'   -- or 'GLUE' for read-only external
--     BASE_LOCATION = 'sales/';
-- (Full setup requires External Volume configuration — concept demo)

-- Clean up
DROP DATABASE IF EXISTS STORAGE_DEMO;
```

☒ Topic 1.5 — 20 Practice Questions

Q1. Which table type provides BOTH Time Travel AND Fail-Safe protection?

- A) Transient
- B) Temporary
- C) Permanent
- D) External

Q2. What is the maximum Time Travel retention for a Transient Table, regardless of Snowflake edition?

- A) 7 days
- B) 3 days
- C) 1 day
- D) 0 days (no Time Travel)

Q3. An External Table in Snowflake is:

- A) A table stored in a Snowflake-managed location but accessible by external systems
- B) A read-only table that points to files in external cloud storage (S3/Blob/GCS) — data is NOT copied into Snowflake
- C) A table that can be written to by both Snowflake and external systems simultaneously
- D) A table stored in Snowflake but shared with a partner account via Secure Data Sharing

Q4. Which Snowflake table type automatically refreshes its content based on a configured TARGET_LAG?

- A) Materialized View
- B) External Table
- C) Transient Table
- D) Dynamic Table

Q5. Which of the following are TRUE about Apache Iceberg tables in Snowflake? *(Select ALL that apply)*

- A) They store data in Apache Parquet format in external cloud storage
- B) When using an external catalog (e.g., Glue), they are read-only in Snowflake
- C) They support full DML (INSERT/UPDATE/DELETE) when using Snowflake as the catalog
- D) They store data in Snowflake's proprietary format
- E) They replace all permanent tables in a Snowflake account automatically

Q6. What is the Fail-Safe period for a Permanent Table?

- A) Same as Time Travel — configurable from 0–90 days
- B) Fixed 7 days — starts AFTER the Time Travel period expires
- C) 24 hours — matches the result cache lifetime
- D) Fixed 30 days for all table types

Q7. A Materialized View in Snowflake is limited to:

- A) Any number of source tables as long as all are in the same schema
- B) A single base table — JOINs between multiple tables are NOT supported
- C) Three base tables maximum

- D) Only tables with clustering keys enabled

Q8. Which Snowflake editions support Materialized Views? *(Select ALL that apply)*

- A) Standard
- B) Enterprise
- C) Business Critical
- D) Virtual Private Snowflake

Q9. A Secure View differs from a Standard View in that:

- A) Secure views store data physically while standard views do not
- B) The SQL definition of a Secure View is hidden from non-owners, and optimizer pushdown is disabled
- C) Secure views require Enterprise edition; standard views require only Standard edition
- D) Secure views auto-refresh like materialized views

Q10. Which table type is appropriate for staging ETL data that needs NO disaster recovery (no Fail-Safe) and saves storage costs?

- A) Permanent Table
- B) Temporary Table
- C) Transient Table
- D) External Table

Q11. What is "partition pruning" in Snowflake?

- A) Automatically deleting old micro-partitions after 30 days
- B) The query optimizer skipping micro-partitions whose min/max column metadata doesn't include the query's WHERE filter value — reducing data scanned
- C) Manually removing partitions with a PRUNE TABLE command
- D) Compressing micro-partitions to reduce storage costs

Q12. Can a Secure View and a Materialized View be combined into one view object?

- A) No — Secure and Materialized are mutually exclusive view types
- B) Yes — `CREATE SECURE MATERIALIZED VIEW` creates a view that is both pre-computed AND hides its definition from non-owners
- C) Yes — but only if you add the SECURE keyword after creating a materialized view
- D) No — Secure views are only for standard (non-materialized) views

Q13. A Temporary Table is accidentally created with the same name as an existing Permanent Table. What does querying that name return in the current session?

- A) An error — duplicate table names are not allowed
- B) The Permanent Table — temporary tables never shadow permanent tables
- C) The Temporary Table — it shadows the permanent table with the same name within the session
- D) Both tables are queried and results are merged

Q14. What does `SYSTEM$CLUSTERING_INFORMATION('my_table', '(my_column)')` return?

- A) The physical location of the micro-partitions for that table and column
- B) Clustering depth metrics — how well micro-partitions are organized by the specified column (lower depth = better clustering = more effective pruning)
- C) The compression ratio of the specified column
- D) The number of distinct values in the specified column

Q15. Which of the following can be included in a Materialized View definition? *(Select the BEST answer based on restrictions)*

- A) `SELECT * FROM table_a JOIN table_b ON table_a.id = table_b.id`
- B) `SELECT dept, COUNT(*), SUM(salary) FROM employees GROUP BY dept`
- C) `SELECT * FROM table_a ORDER BY created_date LIMIT 100`
- D) `SELECT user_id, RANK() OVER (PARTITION BY dept ORDER BY salary) FROM employees`

Q16. What is the main advantage of columnar storage inside micro-partitions?

- A) Write operations are faster because new rows are appended to the end of column files
- B) Analytical queries that reference only specific columns read only those columns' bytes — not all columns — significantly reducing I/O for wide tables
- C) Columnar storage automatically creates indexes on each column
- D) Columnar format allows Snowflake to support row-level updates without creating new partitions

Q17. Automatic Clustering in Snowflake uses what type of compute for its background maintenance?

- A) Virtual Warehouse compute credits (your active warehouse)
- B) Serverless compute credits (managed by Snowflake, billed separately)
- C) Cloud Services layer resources (free, part of 10% threshold)
- D) A dedicated system warehouse that runs automatically

Q18. Which table type is automatically dropped when the user's session ends?

- A) Transient
- B) Temporary

- C) Dynamic
- D) External

Q19. An Iceberg table configured with an EXTERNAL catalog (e.g., AWS Glue) in Snowflake is:

- A) Fully writable — Snowflake can INSERT, UPDATE, and DELETE from it
- B) Read-only in Snowflake — DML is not supported when the catalog is external
- C) Write-only — Snowflake can load data but not read it
- D) Automatically synced with the external catalog on a schedule

Q20. Which view type is REQUIRED when sharing a view through Snowflake's Secure Data Sharing feature?

- A) Standard View
- B) Materialized View
- C) Secure View (or Secure Materialized View)
- D) External Table Views (ETV)

SUBDOMAIN 1.6 — AI/ML and Application Development Features

Layman's Explanation

This is the **newest section of Domain 1** for the COF-C03 exam. Snowflake has transformed from a pure data warehouse into an AI Data Cloud — meaning you can build AI, ML, and full applications inside Snowflake without moving data out.[8][1]

Think of these features as a toolkit:

- **Snowflake Notebooks** = An interactive lab notebook (like Jupyter) that lives inside Snowflake. Write Python and SQL in cells, run them, see charts — all without leaving Snowflake.
- **Streamlit in Snowflake** = A way to build actual web apps (like dashboards or data tools) directly inside Snowflake using Python. Users open a URL and see a live interactive app — no web server needed.
- **Snowpark** = A library for Python, Java, and Scala developers. Instead of extracting data from Snowflake to process it locally (slow, expensive, risky), Snowpark lets you write Python/Java/Scala code that runs INSIDE Snowflake on your data — zero data movement.
- **Snowflake Cortex** = AI built directly into SQL. Ask it to summarize text, translate languages, detect sentiment, or answer questions from your documents — using a single SQL function call. No AI expertise needed.

- **Snowflake ML** = Tools for building traditional machine learning models (classification, regression, anomaly detection, forecasting) entirely within Snowflake.

Technical Explanation

Snowflake Notebooks[^8]

What it is: An interactive cell-based computing environment embedded in Snowsight. Each cell runs SQL or Python independently, with output (tables, charts, logs) displayed inline.

Key features:

- Supports SQL cells AND Python cells in the same notebook
- Uses a Virtual Warehouse for SQL execution; uses a **Snowpark-Optimized warehouse** (or compute pool for Container Runtime) for Python
- Can import Python packages from the Snowflake Anaconda channel
- Supports inline Streamlit widgets for interactive elements within a notebook
- Notebooks can be scheduled like Tasks
- Git integration: notebooks can be stored in a Git repository

Edge cases:

- Each SQL cell uses the current session's warehouse context
- Python cells share a persistent Python kernel within the notebook session
- Not the same as local Jupyter Notebooks — they run inside Snowflake

Streamlit in Snowflake[16][17][^8]

What it is: The ability to deploy and run Streamlit Python applications directly inside a Snowflake account — fully hosted, no external server needed.

Key features:

- Create interactive web apps (dashboards, data exploration tools, ML demos) in Python
- App code runs in Snowflake — data never leaves the platform
- Access data via Snowpark session — no Snowflake credentials stored in the app code
- Deployed via SQL: `CREATE STREAMLIT app_name...` or via Snowflake CLI: `snow streamlit deploy`
- Accessible through a URL within Snowsight

Edge cases:

- Apps run in a **Snowflake-managed compute environment** (not a warehouse)

- Access controlled by Snowflake RBAC — only users with appropriate roles can view/run apps
- Each app has an associated warehouse for any SQL queries it runs

Snowpark[18][19] [20][21]

What it is: A set of libraries and runtimes that allow developers to write code in Python, Java, or Scala that executes INSIDE Snowflake — bringing compute to the data rather than data to the compute.

Supported languages and their main use:

Language	Use Cases
Python	Data engineering pipelines, ML model training, UDFs, stored procedures, DataFrame API
Java	UDFs, stored procedures, JDBC-style operations
Scala	UDFs, stored procedures, Snowpark DataFrame API
JavaScript	UDFs only (no Snowpark library)

Key concepts:

- **Snowpark DataFrame API:** Fluent API to transform data (like PySpark/Pandas) — generates optimized SQL under the hood
- **Snowpark UDFs:** Python/Java/Scala functions deployed to Snowflake, callable from SQL
- **Snowpark Stored Procedures:** Full procedural logic in Python/Java/Scala running inside Snowflake with DML support
- **Vectorized UDFs:** Python UDFs that receive a Pandas Series instead of one row at a time — much faster for row-by-row operations

Edge cases:

- Python Snowpark stored procedures require `snowflake-snowpark-python` library
- Supported Python versions for stored procedures: 3.9 (deprecated), 3.10, 3.11, 3.12, 3.13[^22]
- Scala supported versions: 2.12 and 2.13[^19]
- Snowpark code runs using a warehouse — the Snowpark-Optimized type is recommended for ML workloads

Snowflake Cortex[23][24] [25][26]

What it is: A suite of AI features built directly into Snowflake, accessible via SQL functions. Uses large language models (LLMs) hosted by Snowflake — no external API keys needed.

Cortex AI SQL Functions (Task-Specific)

Function	What It Does	Input
`SNOWFLAKE.CORTEX.AI_COMPLETE(model, prompt)`	General-purpose LLM completion — generate text, classify, answer questions	Text prompt + model name
`SNOWFLAKE.CORTEX.SUMMARIZE(text)`	Summarizes English text into a shorter form	Text column
`SNOWFLAKE.CORTEX.TRANSLATE(text, from_lang, to_lang)`	Translates text between supported languages	Text + language codes
`SNOWFLAKE.CORTEX.SENTIMENT(text)`	Returns sentiment score: -1.0 (negative) to 1.0 (positive)	Text column
`SNOWFLAKE.CORTEX.EXTRACT_ANSWER(question, context)`	Extracts the answer to a question from a given context passage	Question + context text

Important name change: `COMPLETE` was renamed to `AI_COMPLETE` in recent versions. Both may appear on the exam.[24][26]

Cortex Search

A managed search service for unstructured text data. Build a semantic search engine over documents, FAQs, or knowledge bases — returns relevant results using vector similarity (RAG-based).

Cortex Analyst

An AI assistant that translates natural language questions into SQL queries against your structured data — a text-to-SQL capability that understands your data model and business terms.

Edge cases on Cortex:

- All Cortex functions are billed in Snowflake credits (not external API costs)
- SUMMARIZE is a scalar function (one row → one summary); use `AI_SUMMARIZE_AGG` for aggregating multiple rows
- SENTIMENT returns a FLOAT between -1 and 1 (not a category label)
- TRANSLATE supports specific language pairs — not all languages are supported for all directions

Snowflake ML[16][8]

What it is: A set of tools for building, training, and deploying machine learning models entirely within Snowflake.

Components:

Component	Description
ML Functions	Built-in AutoML functions: `FORECAST`, `ANOMALY_DETECTION`, `CLASSIFICATION`, `TOP_INSIGHTS`, `CONTRIBUTION_EXPLORER`
Feature Store	Central repository for storing and versioning ML features
Model Registry	Version, log, and deploy trained ML models
Snowpark ML Library	Python library for data preprocessing and model training (scikit-learn API compatible)

Edge cases:

- ML Functions (FORECAST etc.) are **serverless** — they don't use a warehouse
- Custom model training via Snowpark ML DOES use a warehouse (Snowpark-Optimized recommended)
- Models stored in the Model Registry are schema-level objects

Hands-On Queries: AI/ML and App Features

```
-- ============================================================
-- SETUP
-- ============================================================
USE ROLE SYSADMIN;
CREATE DATABASE IF NOT EXISTS AI_DEMO;
USE DATABASE AI_DEMO;
CREATE SCHEMA IF NOT EXISTS DEMO;
USE SCHEMA DEMO;
USE WAREHOUSE learn_wh;

-- ============================================================
-- PART 1: Snowflake Cortex AI SQL Functions
-- ============================================================

-- Create a product review table
CREATE OR REPLACE TABLE product_reviews (
    review_id   INTEGER,
    product     VARCHAR(100),
    review_text VARCHAR(2000),
    language    VARCHAR(10)
);

INSERT INTO product_reviews VALUES
(1, 'Laptop Pro X',
 'This laptop is absolutely incredible! The performance is blazing fast and the batt
```

```
 'en'),
(2, 'Budget Headphones',
 'The sound quality is terrible and they broke after two weeks. Complete waste of mo
 'en'),
(3, 'Wireless Mouse',
 'Decent mouse for the price. Works fine but nothing special. Average build quality.
 'en'),
(4, 'Smart Speaker',
 'Ce produit est fantastique! La qualité sonore est excellente et l installation est
 'fr'),
(5, 'USB Hub',
 'Solid product, all ports work as expected. Would buy again.',
 'en');

-- SENTIMENT: Score each review (-1 = very negative, +1 = very positive)
SELECT
    review_id,
    product,
    LEFT(review_text, 60)                               AS review_preview,
    SNOWFLAKE.CORTEX.SENTIMENT(review_text)           AS sentiment_score,
    CASE
        WHEN SNOWFLAKE.CORTEX.SENTIMENT(review_text) > 0.3  THEN 'Positive'
        WHEN SNOWFLAKE.CORTEX.SENTIMENT(review_text) < -0.3 THEN 'Negative'
        ELSE 'Neutral'
    END                                                AS sentiment_label
FROM product_reviews
WHERE language = 'en';

-- SUMMARIZE: Get a short summary of a long review
SELECT
    product,
    SNOWFLAKE.CORTEX.SUMMARIZE(review_text) AS short_summary
FROM product_reviews
WHERE review_id = 1;

-- TRANSLATE: Convert French review to English
SELECT
    review_id,
    product,
    review_text                                          AS original_french,
    SNOWFLAKE.CORTEX.TRANSLATE(review_text, 'fr', 'en') AS english_translation
FROM product_reviews
WHERE language = 'fr';

-- EXTRACT_ANSWER: Answer a question from context
SELECT
    SNOWFLAKE.CORTEX.EXTRACT_ANSWER(
        'What does the customer think about battery life?',
        review_text
    ) AS extracted_answer
FROM product_reviews
WHERE review_id = 1;

-- AI_COMPLETE: General-purpose prompt-based generation
SELECT
    product,
```

```
    SNOWFLAKE.CORTEX.AI_COMPLETE(
        'snowflake-llama-3.3-70b',
        'You are a marketing expert. Write a one-sentence promotional tagline for th
    ) AS marketing_tagline
FROM product_reviews
WHERE review_id = 1;

-- ============================================================
-- PART 2: Snowpark Python UDF
-- ============================================================

-- Python UDF: Classify sentiment more precisely
CREATE OR REPLACE FUNCTION py_classify_sentiment(score FLOAT)
RETURNS VARCHAR
LANGUAGE PYTHON
RUNTIME_VERSION = '3.11'
HANDLER = 'classify'
AS $$
def classify(score):
    if score >= 0.6:
        return "Very Positive"
    elif score >= 0.2:
        return "Positive"
    elif score >= -0.2:
        return "Neutral"
    elif score >= -0.6:
        return "Negative"
    else:
        return "Very Negative"
$$;

-- Use the Python UDF with Cortex sentiment scores
SELECT
    product,
    SNOWFLAKE.CORTEX.SENTIMENT(review_text) AS score,
    py_classify_sentiment(SNOWFLAKE.CORTEX.SENTIMENT(review_text)) AS classification
FROM product_reviews
WHERE language = 'en';

-- ============================================================
-- PART 3: Snowpark DataFrame API (Python) — concept in SQL
-- ============================================================
-- In a Snowflake Notebook or Python environment, Snowpark looks like:
--
-- from snowflake.snowpark import Session
-- session = Session.builder.configs(connection_params).create()
--
-- # DataFrame API (like Pandas, but runs inside Snowflake)
-- df = session.table("AI_DEMO.DEMO.PRODUCT_REVIEWS")
-- df_en = df.filter(df["LANGUAGE"] == "en")
-- df_grouped = df_en.group_by("PRODUCT").count()
-- df_grouped.show()
--
-- Everything above runs as SQL INSIDE Snowflake — zero data movement!

-- ============================================================
```

```
-- PART 4: Snowflake ML Functions (Serverless)
-- ============================================================

-- Create a time series for forecasting
CREATE OR REPLACE TABLE daily_sales (
    sale_date DATE,
    sales_amount NUMBER(12,2)
);

INSERT INTO daily_sales
SELECT
    DATEADD('day', seq4(), '2023-01-01')::DATE AS sale_date,
    100 + UNIFORM(0, 50, RANDOM()) +
    (10 * SIN(2 * 3.14159 * SEQ4() / 7)) AS sales_amount  -- weekly seasonality
FROM TABLE(GENERATOR(ROWCOUNT => 365));

-- Create an ML forecasting model (serverless — no warehouse credits!)
CREATE OR REPLACE SNOWFLAKE.ML.FORECAST sales_forecast (
    INPUT_DATA => SYSTEM$REFERENCE('TABLE', 'DAILY_SALES'),
    TIMESTAMP_COLNAME => 'SALE_DATE',
    TARGET_COLNAME => 'SALES_AMOUNT'
);

-- Generate a 30-day forecast
CALL sales_forecast!FORECAST(FORECASTING_PERIODS => 30);
-- Returns: date, forecast, lower bound, upper bound

-- ============================================================
-- PART 5: Streamlit in Snowflake (concept demo)
-- ============================================================
-- A Streamlit app is deployed like this:
-- snow streamlit deploy (from CLI)
-- OR via SQL:
--
-- CREATE STREAMLIT review_analyzer
--     ROOT_LOCATION = '@my_stage/streamlit_app'
--     MAIN_FILE = 'app.py'
--     QUERY_WAREHOUSE = 'learn_wh';
--
-- Example app.py:
-- import streamlit as st
-- from snowflake.snowpark.context import get_active_session
-- session = get_active_session()
-- st.title("Product Review Analyzer")
-- reviews = session.table("AI_DEMO.DEMO.PRODUCT_REVIEWS").to_pandas()
-- st.dataframe(reviews)

-- Clean up
DROP DATABASE IF EXISTS AI_DEMO;
```

▯ Topic 1.6 — 20 Practice Questions

Q1. What is Snowpark?

- A) A visual query builder inside Snowsight for creating complex SQL without typing
- B) A set of libraries for Python, Java, and Scala that allow code to run INSIDE Snowflake — eliminating the need to extract data for processing
- C) A dedicated Snowflake environment for running Apache Spark workloads
- D) A feature that automatically parks (suspends) warehouses after query completion

Q2. Which programming languages does Snowpark support for writing Stored Procedures? *(Select ALL that apply)*

- A) Python
- B) Java
- C) Scala
- D) R
- E) JavaScript

Q3. What does `SNOWFLAKE.CORTEX.SENTIMENT(text)` return?

- A) One of three labels: "Positive", "Negative", or "Neutral"
- B) A FLOAT value between -1.0 (very negative) and 1.0 (very positive)
- C) A JSON object with multiple sentiment dimensions
- D) A percentage score from 0% to 100%

Q4. You want to translate a French customer review to English using Snowflake's built-in AI. Which function do you use?

- A) `SNOWFLAKE.CORTEX.SUMMARIZE(text, 'fr', 'en')`
- B) `SNOWFLAKE.CORTEX.TRANSLATE(text, 'fr', 'en')`
- C) `SNOWFLAKE.CORTEX.AI_COMPLETE('translate ' || text)`
- D) `SNOWFLAKE.CORTEX.EXTRACT_ANSWER('translate to English', text)`

Q5. Snowflake Notebooks support which execution environments? *(Select ALL that apply)*

- A) SQL cells run against the current session's Virtual Warehouse
- B) Python cells run inside Snowflake using Snowpark Python runtime
- C) Notebooks execute on the user's local machine, connecting remotely to Snowflake
- D) Notebooks can use Snowflake Container Runtime for advanced Python workloads

Q6. What is Streamlit in Snowflake?

- A) A SQL-based data streaming service for real-time ingestion

- B) A feature allowing Python-based interactive web applications to be hosted and run directly inside Snowflake — no external server needed
- C) A visual pipeline builder for automating Snowflake Tasks
- D) A Snowflake CLI plugin for deploying applications to AWS

Q7. Which Snowflake Cortex function extracts a direct answer to a natural language question from a provided text context?

- A) `SNOWFLAKE.CORTEX.SUMMARIZE`
- B) `SNOWFLAKE.CORTEX.TRANSLATE`
- C) `SNOWFLAKE.CORTEX.EXTRACT_ANSWER`
- D) `SNOWFLAKE.CORTEX.SENTIMENT`

Q8. What is Cortex Analyst?

- A) A tool that analyzes the execution plan of Snowflake queries
- B) An AI feature that translates natural language questions into SQL queries against your structured data — text-to-SQL capability
- C) A dashboard tool for monitoring Cortex AI function credit usage
- D) An ML model that classifies Snowflake user activity for security auditing

Q9. What is the PRIMARY benefit of running Snowpark code inside Snowflake versus running the same Python code locally?

- A) Snowpark is faster because Python runs at lower level than SQL
- B) Data never leaves Snowflake — no data extraction needed, reducing latency, egress costs, and security risk
- C) Snowpark Python supports more libraries than standard Python
- D) Local Python code cannot access Snowflake data at all

Q10. Which Snowflake ML component provides pre-built AutoML functions for FORECAST, ANOMALY_DETECTION, and CLASSIFICATION?

- A) Snowpark ML Library
- B) Snowflake ML Functions (serverless)
- C) Cortex Search
- D) Model Registry

Q11. You want to summarize 10,000 individual customer reviews into a SINGLE aggregate summary. Which Cortex function applies?

- A) `SNOWFLAKE.CORTEX.SUMMARIZE` (scalar)
- B) `SNOWFLAKE.CORTEX.AI_COMPLETE` with a GROUP BY
- C) `AI_SUMMARIZE_AGG` (aggregate version of SUMMARIZE for multiple rows)

- D) `SNOWFLAKE.CORTEX.EXTRACT_ANSWER` with a summary prompt

Q12. Which warehouse type is RECOMMENDED for training ML models using Snowpark Python?

- A) Standard Gen 1 (most memory-efficient)
- B) Standard Gen 2 (current default)
- C) Snowpark-Optimized (16× more memory per node)
- D) Multi-cluster warehouse (maximum concurrency)

Q13. What does `SNOWFLAKE.CORTEX.AI_COMPLETE(model, prompt)` do?

- A) Automatically completes partial SQL queries
- B) Sends a custom prompt to a large language model and returns a generated text response — general-purpose AI text generation
- C) Completes a data pipeline by triggering a downstream task
- D) Validates and completes partial JSON data structures

Q14. Snowflake ML Functions such as FORECAST are billed using what type of compute?

- A) Virtual Warehouse credits (your active warehouse)
- B) Cloud Services credits (free up to 10% threshold)
- C) Serverless compute credits (managed by Snowflake, separate from warehouse)
- D) External API credits (billed directly by the model provider)

Q15. What is the Snowflake Model Registry?

- A) A Snowflake Marketplace listing of pre-built AI models from third parties
- B) A schema-level object that stores, versions, and manages trained ML models for deployment
- C) A system view showing all SQL UDFs in the account
- D) A Cortex service that registers natural language queries for reuse

Q16. Which statement about Streamlit in Snowflake is TRUE?

- A) It requires an external server (AWS EC2, Azure VM) to host the application
- B) App code runs inside Snowflake, data accessed via Snowpark session — no credentials stored in app code
- C) Streamlit apps bypass Snowflake's RBAC — all users can access all apps
- D) Streamlit apps can only be deployed using the Snowflake CLI — not via SQL

Q17. What is the difference between Cortex Search and Cortex Analyst?

- A) Cortex Search finds SQL queries; Cortex Analyst finds documents

- B) Cortex Search is semantic search over unstructured documents (RAG-based); Cortex Analyst translates natural language questions into SQL over structured data
- C) Cortex Search is available in Standard edition; Cortex Analyst requires Enterprise
- D) They are the same feature with different UI access points

Q18. A Python UDF created with Snowpark differs from a JavaScript UDF in that:

- A) Python UDFs can only be called from Tasks; JavaScript UDFs can be called in any SQL
- B) Python UDFs can use third-party packages from the Snowflake Anaconda channel; JavaScript UDFs cannot use external packages
- C) JavaScript UDFs require Enterprise edition; Python UDFs work on Standard
- D) Python UDFs can execute DML inside UDF code; JavaScript UDFs cannot

Q19. Which Snowflake feature enables building a semantic search engine over unstructured text (like support tickets or documentation) that returns the most relevant results to a user's query?

- A) Snowflake ML Forecasting
- B) Cortex Analyst
- C) Cortex Search
- D) Snowpark Vectorized UDFs

Q20. Which of the following are TRUE about Snowflake Notebooks? *(Select ALL that apply)*

- A) They support both SQL and Python cells in the same notebook
- B) They execute on the user's local Jupyter server connected to Snowflake via JDBC
- C) They can be scheduled to run automatically like Snowflake Tasks
- D) Python cells use the Snowpark Python runtime inside Snowflake
- E) They support Git integration for version control

✅ COMPLETE ANSWER KEY

Topic 1.1 — Snowflake Architecture

Q	Answer	Key Reason
1	A	Snowflake combines shared-disk (shared storage) with shared-nothing (independent compute clusters)[^3]
2	C	Data lives in cloud object storage (S3/Azure Blob/GCS) depending on the deployment region[^4]
3	D	Snowflake uses its own proprietary columnar format — NOT Parquet, ORC, or Avro[^3]
4	A	Only Virtual Warehouse compute credits stop on suspension. Storage and Cloud Services continue billing[^1]

Q	Answer	Key Reason
5	C	COUNT(*) is answered from Cloud Services metadata (row count is always tracked) — no warehouse needed[^3]
6	B	Only 1 physical copy exists in the shared storage layer — all warehouses access the same data[^27]
7	B	MPP = Massively Parallel Processing — multiple nodes divide and conquer a query simultaneously[^3]
8	B, C, D	Cloud Services does: query parsing/optimization ✅, auth/RBAC ✅, metadata ✅. Physical storage ❌ and SQL computation ❌ are other layers[^6]
9	C	Storage is completely independent — dropping a warehouse has zero effect on data[^27]
10	C	COUNT(*) uses row-count metadata — no warehouse. SUM/AVG/WHERE need data scanning → need warehouse[^3]
11	C	AES-256 at rest means all stored data is always encrypted, always automatic — no opt-in needed[^1]
12	B	NVMe SSD cache is completely lost when warehouse suspends — nodes are returned to cloud provider[^28]
13	A, B, C	AWS, Azure, GCP are supported. IBM Cloud and Oracle Cloud are not[^29]
14	B	Cloud Services runs continuously as a fully managed service — no warehouse dependency[^6]
15	B	Cloud Services is free up to 10% of daily compute credits — excess above 10% is billed[^1]
16	B	SUM must scan all price values — not in metadata. COUNT(*) only needs row count which IS in metadata[^3]
17	B	MVCC = Multi-Version Concurrency Control, implemented in Cloud Services for concurrent access without blocking[^6]
18	B	Columnar — each column's data is stored together within each micro-partition[^4]
19	A, C, D	50–500 MB ✅; Immutable (DML creates new) ✅; Min/max/count/null metadata ✅. Users can't assign rows manually ❌; Format is proprietary not Parquet ❌[3][4]
20	B	Separation of storage and compute lets you run two completely independent warehouses against the same data — ETL and reporting don't affect each other[^27]

Topic 1.2 — Interfaces and Tools

Q	Answer	Key Reason
1	B	Snowsight is the primary browser-based UI at app.snowflake.com[^7]
2	B	Snowflake CLI's `snow stage put` is designed for scripting and automation[^8]
3	B	Query Profile shows the visual execution plan — partitions scanned, bytes from cache, spill to disk[^8]
4	B	`snow sql -q "..."` executes SQL from terminal and returns results to stdout[^8]
5	C	Classic Console is the deprecated old UI — Snowsight replaced it[^9]

Q	Answer	Key Reason
6	C	Snowsight Dashboards combine SQL chart tiles into visual reports — no coding needed[^7]
7	A, B, C, D	CLI supports: SQL execution, Streamlit deploy, stage upload, Cortex commands. It cannot replace Cloud Services[^8]
8	B	CLI config is in `~/.snowflake/config.toml`[^8]
9	A, B, D	VS Code extension: browse objects ✅, run SQL with syntax highlight ✅, connect with credentials ✅. Doesn't replace warehouses ❌[^8]
10	B	Query History shows elapsed time, bytes scanned, partitions, warehouse, status — not physical file names or IP addresses[^7]
11	B	Snowflake CLI (`snow` command) replaced SnowSQL as the recommended CLI tool[^8]
12	B	Snowflake CLI is the right tool for CI/CD scripted automation[^8]
13	A, B	Snowsight Notebooks support SQL and Python cells. R and Java are not supported in Notebooks[^8]
14	C	The Admin tab in Snowsight manages users, roles, warehouses, billing, etc.[^7]
15	B	Sharing a worksheet gives the colleague access to the SQL text — they run it with their own role and context[^7]
16	B	`config.toml` stores named connection profiles with account, username, auth method for different environments[^8]
17	B	`snow cortex summarize --file` sends a local text file to Cortex AI for summarization[^23]
18	C	Snowflake Notebooks provide interactive Python/SQL cell-by-cell execution inside Snowflake[^8]
19	B	The Data tab in Snowsight lets you browse the object hierarchy and inspect schemas, tables, columns without SQL[^7]
20	B	Snowflake provides ODBC/JDBC drivers and native connectors for BI tools like Tableau and Power BI[^1]

Topic 1.3 — Object Hierarchy and Types

Q	Answer	Key Reason
1	B	Org → Account → Database → Schema → Object is the correct hierarchy[^1]
2	A, B, D, F	Warehouses ✅, Users ✅, Resource Monitors ✅, Network Policies ✅ are account-level. Tables ❌ and Sequences ❌ are schema-level[^1]
3	C	PUBLIC and INFORMATION_SCHEMA are auto-created in every database[^1]
4	C	Three-part name = `DATABASE.SCHEMA.OBJECT` → `COMPANY_DB.SALES.ORDERS`[^1]
5	B	User stages are auto-created for every user and cannot be dropped[^1]
6	B	UDFs return values and are used in SQL; Stored Procedures are called with CALL and can run DML[^20]

Q	Answer	Key Reason
7	D	Object level overrides all others — highest priority in the parameter hierarchy[^1]
8	B	INFORMATION_SCHEMA is a read-only, real-time metadata view of all objects in the current database[^1]
9	D	Named External stages require a Storage Integration (account-level IAM object) to connect to S3/Blob/GCS[^1]
10	B	NEXTVAL returns the next unique sequential integer from the sequence[^1]
11	B	Database Roles are scoped to one database; they can be shared with consumer accounts — Account Roles span the whole account[^1]
12	B	Object-level (table's setting = 30 days) wins over account-level (7 days)[^1]
13	B	File Format objects store named definitions of file structure — reused in COPY INTO and external tables[^1]
14	B	User and Table stages are auto-managed and CANNOT be dropped[^1]
15	B	One database per domain, each with its own schemas — the recommended pattern for multi-domain accounts[^1]
16	A, B, C, D, E	Snowflake UDFs support: JavaScript, Python, Java, Scala, SQL. R is not supported[^20]
17	B	`SET var = value` creates a session variable. `$var` references it in SQL — scoped to current session only[^1]
18	B	`UNDROP SCHEMA schema_name` restores a dropped schema and all objects within it[^1]
19	C	Directory Tables (enabled on named stages) provide a `DIRECTORY(stage_name)` view of file metadata[^1]
20	B	CURRVAL returns the most recent value produced by NEXTVAL in the current session[^1]

References

1. Free Practice Questions for Snowflake COF-C03 Certification - May 3rd, 2026. Study with 563 exam-style practice questions designed to help you prepare for the Sno...
2. Snowflake SnowPro Core Certification Exam Syllabus - It is recommended for all the candidates to refer the COF-C03 objectives and sample questions provid...
3. Day 2: Snowflake's 3-Layer Architecture | SnowPro 50 Days - Yesterday you learned what Snowflake is. Today you learn how it works under the hood. Snowflake's 3-...
4. Snowflake Architecture: A Technical Deep Dive into Cloud Data ...www.datacamp.com › blog › snowflake-architecture - Explore Snowflake's three-layer architecture, data warehouse design, and advanced features. Learn ho...
5. Snowflake 3-Layer Architecture Complete Guide 2026 | Storage + Compute + Services | SnowPro Core - ❄ *Snowflake Compute Layer + Services Layer + Editions Full Explained 2026 | SnowPro Core COF-C03...

6. 1.1 Describe and use the Snowflake architecture (COF-C03) - the official SnowPro Core certification roadmap. Breakdown of the exam structure, focusing on key do...
7. Snowsight: The Snowflake web interface - Snowsight provides a unified experience for working with your Snowflake data by using SQL or Python:...
8. Snowflake SnowPro Core (COF-C03) Full Course: Pass the Exam in 1 Video! [7 Hours] - Master the Snowflake SnowPro Core (COF-C03) exam in this 7-hour complete masterclass! This is the on...
9. SnowPro Core Certification Tutorial (COF-C03)(Unofficial) - YouTube - Pass your SnowPro Core COF-C03 exam on the first try! ❄ Today we master Domain 1.2 Use Snowflake In...
10. Apache Iceberg in Snowflake: A Practical Guide [2025] - Learn how Snowflake works with Apache Iceberg and why a metadata control plane is key for unlocking ...
11. Snowflake & Apache Iceberg: Enterprise Cloud Data Warehouse | OLake - Explore OLake's insights on Snowflake's native Iceberg catalog, automatic optimization, Snowpipe str...
12. Apache Iceberg™ tables | Snowflake Documentation
13. DAY 12: View Types: Standard, Materialized, Secure | SnowPro 50 ... - Yesterday you met Snowflake's six table types and learned how to match the right type to the right w...
14. Snowflake Views: Types and Use Cases | PDF - The document explains the concept of views in Snowflake, which are virtual tables that simplify comp...
15. Snowflake's type of tables and views - what is the difference? - Snowflake's type of tables and views. Snowflake present us seeral options of tables and vies. Let's ...
16. Exploring Snowflake Cortex AI with Snowpark and Streamlit - Revefi - The session explored practical applications of Snowflake Cortex AI and how to use Snowpark and Strea...
17. Build and deploy Snowpark ML models using Streamlit ... - Snowflake - In this quickstart, you will be introduced to ML Sidekick, a no-code app built using Streamlit in Sn...
18. Snowflake + Snowpark Python = machine learning? - dataroots - Snowpark is the Snowflake SDK. It's available for Java, Javascript, Scala and (recently) Python! For...
19. Writing Scala handlers for stored procedures created with SQL - You can create a stored procedure whose handler is written in Scala. You can use the Snowpark librar...
20. Data Type Mappings Between SQL and Handler Languages
21. Snowpark Developer Guide for Scala - Snowflake Documentation - Use the Snowpark API to call system-defined functions, UDFs, and stored procedures. A Simple Example...
22. Writing stored procedures with SQL and Python - You can write a stored procedure whose handler is coded in Python. By using APIs from the Snowpark l...
23. Snowflake Cortex AI Functions (including LLM functions)
24. Snowflake Cortex LLM functions: A complete overview (2026) - Snowflake Cortex LLM Functions—Easily analyze text, ...
25. Snowflake Cortex AISQL Functions - Explore how Snowflake Cortex AISQL functions bring the power of generative AI directly into SQL with...

26. Task-Specific AI SQL You Must Know for the Exam - YouTube - In this video, you'll learn why Task-Specific AI functions in Snowflake Cortex are important and how...
27. Snowflake Architecture Finally Explained Simply (COF-C03 ... - Reddit - Just published a deep dive on Snowflake Architecture for the SnowPro Core (COF-C03) certification ex...
28. Snowflake query optimization: A comprehensive guide ... - Improve your Snowflake queries' performance and efficiency with these proven Snowflake query optimiz...
29. SnowPro® Core Certification (COF-C03) - Snowflake University - Use the Snowflake AI Data Cloud architecture · Manage Snowflake accounts and virtual warehouses · Pe...